



東華大學
DONGHUA UNIVERSITY

ICPC / CCPC
Standard Code Library

Gospel_rock



Coach

老黃暢談

Contestants

ZnZrYb

xixii_

47523

Build Time: August 19, 2026 by ZnZrYb

Contents

1	梦的开始	7
1.1	ASCII 表	7
1.2	dx dy 4	7
1.3	一维数组二维化	7
1.4	初始化模板	7
1.5	CMake 配置	8
1.5.1	Sanitizer 小技巧	8
1.5.2	Cmake Release 配置小技巧	8
1.5.3	小心 vector 大小等相关调试信息信息显示错误	8
1.6	本地手测时怎么发 EOF	8
1.7	CLion 常用功能与快捷键 (默认 keymap)	9
1.8	CLion 语言引擎: 赛场机上先切 Nova	9
1.9	Windows: 禁用 Shift 切换中英文输入	9
1.9.1	微软拼音 (系统自带, Win10 逐步截图)	10
1.9.2	搜狗输入法	12
2	基础数学	13
2.1	快速幂	13
2.2	矩阵快速幂	13
2.2.1	矩阵快速幂加速递推	14
2.3	自动取模类	15
2.4	异或线性基 (linear basis)	16
2.5	带权 bitset 线性基 (F2 线性方程组求解)	17
2.6	区间异或线性基 (prefix linear basis)	18
2.7	n 进制异或线性基	19
2.8	MEX (Minimum Excluded Value)	20
2.9	中位数 (Median)	20
2.9.1	中位数的双向判定不等式	20
2.9.2	± 1 编码 + 前缀和: $O(1)$ 区间中位数判定	20
2.9.3	多段奇长度划分公共中位数 = 全局中位数	20
2.9.4	L_1 最优化是中位数 (对比 L_2 平方和最优化是平均数)	21
2.9.5	流式 / 增量中位数: 对顶堆	21
2.9.6	区间第 k 大 (中位数是 $k = \lceil \text{len}/2 \rceil$ 的特例)	21
2.10	常见结论 (conclusion)	21
2.10.1	约数个数定理	21
2.10.2	Legendre 公式 ($n!$ 的质因子指数)	21
2.10.3	二次方程根与二阶线性递推	22
2.10.4	逆序对奇偶性与置换环	22
3	数论	23
3.1	线性筛	23
3.2	Miller-Rabin 与 Pollard's Rho	23
3.3	欧拉定理, 扩展欧拉定理, 欧拉函数的求解 (线性筛, 试除法)	24
3.3.1	欧拉定理与费马小定理的证明	25
3.4	gcd (迭代与递归)	25
3.5	exgcd 与逆元	26
3.6	在线快速逆元 ($O(1)$ 查询)	26
3.7	中国剩余定理 (CRT)	27
3.7.1	CRT 双射形式 (一维循环 $\leftrightarrow$ 二维网格)	27
3.7.2	扩展中国剩余定理 (exCRT)	27
3.7.3	CRT 的典型应用	28
3.8	裴蜀定理 (Bézout's identity)	28
3.8.1	裴蜀定理的典型应用	28
3.9	莫比乌斯反演 (Möbius inversion)	29
3.9.1	莫比乌斯函数 μ	29
3.9.2	反演公式	29
3.9.3	关键引理: $\sum_{d n} \mu(d) = [n = 1]$	29

3.9.4	反演公式（因数形式）的证明	30
4	计数与组合	32
4.1	Catalan 数（合法括号串数量）	32
4.2	求组合数（递推法）	32
4.3	求组合数（逆元）	32
4.4	插板法（stars and bars）求非负整数解的数量	33
4.5	Lucas 定理	33
4.6	组合相关公式以及技巧	33
4.6.1	离散变量概率变期望	33
4.6.2	朱世杰恒等式（Hockey-stick identity）	34
4.7	Hook-length 公式（树版本）：递增标号计数	34
4.8	Erdős-Szekeres 定理（单调子序列）	34
5	博弈论	36
5.1	Nim 博弈	36
5.1.1	三角形博弈（带约束的 Nim）	36
5.1.2	Moore's Nim-K（每回合可同时动 K 堆）	37
5.1.3	反 Nim（Anti-Nim / Misère Nim）	37
5.1.4	阶梯 Nim	37
5.1.5	Lasker's Nim（可拆分的 Multi-SG）	37
5.2	巴什博弈（单堆有上限取）	37
5.3	SG 函数的暴力打表与 Anti / Every 变体	37
5.3.1	反 SG（Anti-SG）	38
5.3.2	Every-SG（每回合走”所有还能动的石子”）	38
5.3.3	实战范例：CCPC 2025 网络赛 F「连线博弈」——XOR 哈希切子游戏	38
5.4	威佐夫博弈（双堆联动）	39
5.5	斐波那契博弈（前手两倍约束）	39
5.6	图上删边游戏	39
5.6.1	树上删边（Colon Principle）	39
5.6.2	无向图删边（Fusion Principle）	39
6	进阶数学	40
6.1	FFT 多项式乘法（非递归）	40
6.1.1	高精度乘法（基于 FFT）	40
6.1.2	FFT 的使用方法与常见应用	40
6.2	线性规划（两阶段单纯形 Simplex）	42
7	位运算相关恒等式	45
7.1	加法与位运算转换	45
7.2	或、与、异或的关系	45
7.3	减法与位运算	45
7.4	& 与 的分配律	46
7.5	lowbit 的推导	46
7.6	去除最低位的 1	46
7.7	连续异或前缀和	46
7.8	对相邻元素异或的操作转化为前缀异或交换	46
7.9	集合内两两最小异或和	47
8	基础算法	48
8.1	二维前缀和	48
8.2	二维差分	48
8.3	双指针模板	48
8.4	整数二分	49
8.4.1	partition_point 实现二分答案（C++20）	49
8.5	三分（实数）	49
8.6	三分整数	50
8.7	倍增法	50
8.8	LIS 最长上升子序列及其还原	50

8.8.1	Dilworth 定理 (最小链覆盖 / 偏序划分)	51
9	数据结构	52
9.1	单调栈	52
9.2	离散化	52
9.3	ST 表 (Sparse Table)	53
9.4	树状数组 (BIT / Fenwick Tree)	54
9.4.1	单点加, 区间求和	54
9.4.2	区间加, 单点查询	55
9.4.3	区间加, 区间求和	55
9.4.4	单点改, 区间最值查询 (BIT 实现 RMQ)	56
9.5	珂朵莉树 (ODT / Chtholly Tree)	57
9.6	懒标记线段树 (Tag + Info 写法)	58
9.7	带权可撤销并查集 (判二分图)	59
9.8	线段树分治	60
9.9	zkw 线段树	60
9.9.1	zkw 线段树区间加	60
9.9.2	线段树区间加乘 (递归 / zkw)	61
9.9.3	zkw 区间推平 $+-$, 区间和查询	62
9.9.4	zkwlen	63
9.9.5	线段树 RMQ (zkw / 递归式)	63
9.9.6	zkw 小白逛公园 (维护最大子段和)	64
9.9.7	zkw 线段树维护平方和	65
9.9.8	zkw 维护立方和, 四则运算, 区间推平	66
9.10	segSum (最普通的线段树) (返回第 k 个 1 的下标) (线段树二分)	67
9.11	segMxPosIntervalAdd 区间查询最大值位置以及最大值	69
9.12	segModify (扶苏的问题) (区间查询最大, 最小值, 区间赋值, 区间 $+-$)	71
9.13	segMul (区间 $+-$, 区间乘法)	73
9.14	segMxSum (小白逛公园) (最大子段和)	75
9.15	动态开点线段树 (区间加, 区间求和)	76
9.16	线段树上二分	77
9.16.1	前缀含负数时找区间第一个前缀和 $\geq x$ 的位置	78
9.17	segValueModAdd 求全局最小众数的权值线段树	78
9.18	线段树可视化	80
9.19	pb_ds 红黑树 (tree)	81
9.20	gp_hash_table (哈希表, 比 unordered_map 快)	82
9.21	rope ($O(1)$ 拷贝的可持久化序列)	83
9.22	FHQ-treap (无旋 Treap)	83
9.23	替罪羊树 (Scapegoat Tree)	84
9.24	主席树 (可持久化权值线段树)	86
9.25	分块 (单点修改)	87
9.26	回滚莫队	88
10	图论	89
10.1	倍增法求 LCA	89
10.2	虚树 (Virtual Tree)	89
10.3	并查集 (DSU) 与 DSU on next 技巧	91
10.3.1	带权并查集 (class 封装, 维护到根距离与连通块大小)	92
10.3.2	带权并查集 (群作用泛型版) + 不动点钉死 pinned_dsu	93
10.4	Borůvka 最小生成树	96
10.5	树上背包	96
10.6	Floyd 全源最短路	97
10.7	Dijkstra (堆优化)	97
10.8	二分图判定 (染色法)	97
10.9	匈牙利算法 (二分图最大匹配)	98
10.9.1	Hall 婚配定理 (完美匹配判定)	98
10.10	差分约束	98
10.11	Tarjan 缩点为边双连通分量 (EBCC)	99

10.12	Tarjan 求强连通分量 (SCC)	99
10.13	点分治	100
10.14	重链剖分 (HLD + 线段树)	101
10.15	欧拉回路与欧拉路径	103
10.16	Color Coding (随机染色 + 状压 DP)	103
10.16.1	转移骨架	103
10.16.2	剪枝技巧	104
10.16.3	复杂度	104
10.16.4	一句话总结	104
10.17	合法括号序列 $\leftrightarrow$ 有根有序树	104
10.18	网络流 (Dinic): 最大流 / 最小割容量及割边集合	105
10.19	最小费用最大流 (SPFA 势能 + Dijkstra 增广)	106
10.20	图论常见结论	108
10.20.1	树直径端点的“远点”性质	108
10.20.2	相邻点单源最短路差不超过 1	108
10.20.3	广义凯莱公式 (森林连通块版)	108
10.20.4	两条树上路径是否相交 (LCA 判定)	108
11	字符串算法	109
11.1	Manacher 算法	109
11.2	Z 函数 (扩展 KMP)	109
11.3	KMP (前缀函数 pi)	110
11.4	字符串哈希 (mod $2^{61} - 1$ 版本)	110
11.5	子序列自动机	111
11.6	后缀自动机 (SAM)	112
11.7	Trie 树	113
11.8	01 Trie (XOR Trie)	113
11.9	字符串分词与输出技术 (大模拟读入)	115
12	动态规划 (dp)	116
12.1	二进制子集枚举技巧	116
12.1.1	枚举某个集合的所有子集	116
12.1.2	枚举固定大小的子集	116
12.1.3	枚举补集元素做前向转移 (附「交判子集」)	116
12.2	SOS DP (Sum / Max over Subsets, 高维前缀和)	117
12.2.1	典型应用: 按位无交最大值	117
12.3	子集格最短路 (超集和 + 状压 DP 求最优排列)	117
12.4	线段树维护 dp 求最大独立集	119
13	计算几何	121
13.1	点线基础模板	121
13.2	点在线上的投影点	124
13.3	对称点	124
13.4	点线位置判断 (CCW)	125
13.4.1	逆时针弧角 ccw_angle (标量 / 向量两版)	125
13.5	判断两直线是否垂直 / 平行	126
13.6	判断两线段是否相交	126
13.7	两条直线交点	126
13.8	两线段间最小距离	127
13.9	多边形面积	128
13.10	圆 Circle: 测度五件套	128
13.11	两圆位置关系 circle_relation	130
13.12	三角形内接圆 / 外接圆	130
13.13	直线与圆的关系 / 交点 circle_line_relation / getCrossPointsCL	131
13.14	两圆的关系 / 交点 circle_circle_relation / getCrossPointsCC	132
13.15	过一点圆的切点 getTangentPoints	133
13.16	两圆公切线 getCommonTangentPoints	133
13.17	两圆交集面积 two_circle_intersect_area	134
13.18	简单多边形与圆的交集面积 polygon_circle_intersect_area	134

13.19 多边形凸性检验	136
13.19.1 反例 1: 内角为反向凹角 (外角为锐角)	136
13.19.2 反例 2: 自交多边形 (蝴蝶结)	136
13.19.3 反例 3: 内角恰为 180°	136
13.20 点与多边形关系	137
13.21 极角排序	138
13.22 Minkowski 闵可夫斯基和	139
13.23 半平面交 (half_plane_cut)	139
13.24 凸包 (Andrew 算法)	141
13.24.1 对偶点技巧: 直线上半包络 $\Leftrightarrow$ 对偶点上凸壳	142
13.25 凸多边形直径 (旋转卡壳)	142
13.26 最远点对 (旋转卡壳, 返回原始下标)	144
13.27 最小矩形覆盖 (旋转卡壳, 4 指针)	145
13.28 平面最近点对	146
13.29 两凸多边形最近距离 (旋转卡壳)	147
13.30 常用公式	149
13.30.1 正多边形面积公式	149
13.30.2 三角恒等式	149
13.31 附录: C++14 版本基础模板	150
13.32 结论	153
13.32.1 Pick 定理	153
14 STL 及其使用	155
14.1 gp_hash_table (pb_ds 哈希表, 替 unordered_map)	155
14.2 string::find 的起始位置参数与 string_view	155
14.3 mt19937_64 套 splitmix64 抗 BM 攻击	155
14.4 unordered_map 自定义 hash & 结构体做 key	155
14.5 views::keys / views::values 遍历 map 的键 / 值 (C++20 ranges)	156
14.6 sort + unique 按 key 去重并保留指定值	156
14.7 lower_bound / upper_bound 自定义比较器参数顺序相反	157
14.8 __int128 读入输出 (重载运算符版)	157
14.9 next_permutation 枚举全排列	158
15 常见问题与错误	159
15.1 -O2 下的 FMA 乘加融合踩崩浮点 ceil	159
15.2 __float128 使用要点	159
15.3 DP 不可达状态必须用 INF/-INF 显式标识	160
15.4 大网格 DFS 遇到障碍要跳过, 不要继续标记递归	160
16 debug 技巧	161
16.1 除了打印之外, 善用 assert	161
16.1.1 增量 debug	161
16.2 std::string 条件断点的陷阱	161
17 环境测试	162
17.1 试机赛与纪律	162
17.2 编译器版本与扩展头	162
17.3 __int128, __float128, long double 实测	162
17.4 评测机限制与编译选项	162
17.5 #pragma 开优化 / SIMD 实测	163
17.6 性能基准 (map 跑分)	163
18 常用 Python 命令	164
18.1 启动交互解释器	164
18.2 当计算器: 科学计数法速估	164
19 常用技巧 (卡常, 对拍, 打表, 调试等)	165
19.1 快读快写	165
19.1.1 读入, 输出科学计数法小数	165

19.2 开大栈空间	166
19.3 C++ 中 <code>struct</code> 自定义比较器与 <code>std::array</code> 性能对比	166
19.4 计时	166
19.5 对拍	167
19.5.1 Special Judge (答案不唯一) 题目怎么对拍	167
19.6 打表	168
19.6.1 打表中需要输出的东西 (答案)	168
19.6.2 打表中需要输出的东西 (前缀答案)	169
19.7 凸包数据生成	169
19.8 编译 shell	170

1 梦的开始

1.1 ASCII 表

!	33	-	45	9	57	E	69	Q	81	]	93	i	105	u	117
"	34	.	46	:	58	F	70	R	82	^	94	j	106	v	118
#	35	/	47	;	59	G	71	S	83	_	95	k	107	w	119
\$	36	0	48	<	60	H	72	T	84	`	96	l	108	x	120
%	37	1	49	=	61	I	73	U	85	a	97	m	109	y	121
&	38	2	50	>	62	J	74	V	86	b	98	n	110	z	122
'	39	3	51	?	63	K	75	W	87	c	99	o	111	{	123
(	40	4	52	@	64	L	76	X	88	d	100	p	112		124
)	41	5	53	A	65	M	77	Y	89	e	101	q	113	}	125
*	42	6	54	B	66	N	78	Z	90	f	102	r	114	~	126
+	43	7	55	C	67	O	79	[	91	g	103	s	115		
,	44	8	56	D	68	P	80	\	92	h	104	t	116		

1.2 dxdy 4

```
/* -> (Right, Down, Left, Up)
| /
<-
*/
int dx[4] = {1, 0, -1, 0};
int dy[4] = {0, 1, 0, -1};
```

1.3 一维数组二维化

```
// 如果 n * m <= 1e6, 那么应该开 3e6
↳ (如果一行为M+2)
static constexpr ll MAXN = (1ll)3e6 + 10
struct GRID {
    ll a[MAXN] = {};

    ll *operator[](int i) {
        return a + i * (M + 2);
    }
} grid;
```

1.4 初始化模板

```
// teamname: Gospel_rock
/**
 * Problem: ${PROBLEM_NAME}
 * Contest: ${PROBLEM_GROUP_NAME}
 * Judge: ${ONLINE_JUDGE_NAME}
 * URL: ${PROBLEM_URL}
 * Created: ${YEAR}-${MONTH}-${DAY}
↳ ${HOUR}:${MINUTE}:${SECOND}
 * Author: Gospel_rock
 * My blog: https://znzryb.com/
 *
 * Powered by AutoCp
↳ https://github.com/Pushpavel/AutoCp
```

```
*/
#include <bits/stdc++.h>
#define all(vec) vec.begin(), vec.end()
#define lson(o) (o << 1)
#define rson(o) (o << 1 | 1)
#define SZ(a) ((long long)a.size())
#define fsp(x) fixed << setprecision(x)
#define debug(x) cerr << "[" << #x << "]" << ":
↳ " << x << "\n"
#define cend cerr << "\n-----"
↳ -----\n"
#define cEnd cerr << "\n*****"
↳ *****\n"

using namespace std;

using ll = long long;
using ull = unsigned long long;
using DB = long double;
using i128 = __int128;
using CD = complex<double>;

static constexpr ll MAXN = (1ll)1e6 + 10, INF =
↳ (1ll << 61) - 1;
static constexpr ll mod = 998244353; //
↳ (1ll)1e9+7;
static constexpr double eps = 1e-8;
// 不用 M_PI: 后者是 POSIX 扩展, glibc <cmath>
↳ 严格模式会 undef
const long double PI = acosl(-1.0);

ll lT, testcase;

/*
*
*/
```



```

11 N;

void Solve() {
    cin >> N;
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
#ifdef LOCAL
    cout.setf(ios::unitbuf); // 无缓冲流,
    ↪ 方便我们调试
#endif

    cin >> lT;
    for (testcase = 1; testcase <= lT; ++testcase)
        Solve();
    return 0;
}

/*
*/

```

这个 debug(x) 是简版宏：单参数、直接 cerr << x, 要求 x 支持 operator<< (基本类型 / string 都 OK, vector<T> 等容器不能直接打)。不再做 LOCAL/非 LOCAL 切换——OJ 上 stderr 通常不影响判分, 留着也无伤大雅; 介意的话提交前手动把 debug(...) 行删了或包成 #ifdef LOCAL 即可。容器调试请用 range-for 手写循环。

1.5 CMake 配置

1.5.1 Sanitizer 小技巧

有些越界或未定义行为问题, 仅开 address 不一定能完整暴露。实践中建议同时开启 address + undefined, 并保留帧指针, 方便定位调用栈。

```

cmake_minimum_required(VERSION 3.31)
project(Edu_CF_186)

set(CMAKE_CXX_STANDARD 20)

add_executable(Edu_CF_186 main.cpp)

add_compile_definitions(LOCAL)
add_compile_options(-fsanitize=address,undefined)
↪ d -fno-omit-frame-pointer)
add_link_options(-fsanitize=address,undefined)

# 这个不要随便启用, 启用后可能 debug 示数不可信
# add_compile_definitions(_GLIBCXX_DEBUG)

add_executable(A_New_Year_String
    ↪ "src/A_New_Year_String.cpp")
add_executable(B_New_Year_Cake
    ↪ "src/B_New_Year_Cake.cpp")
add_executable(odt_callback_bench "speed
    ↪ test/odt_callback_bench.cpp")

```

```

target_compile_options(odt_callback_bench
    ↪ PRIVATE "SHELL:${CMAKE_CXX_FLAGS_RELEASE}")

```

1.5.2 Cmake Release 配置小技巧

这个写法的主要目的有两个：一是只给特定目标启用 Release 进行性能测试, 二是避免在 GUI 里把全局配置切到 Release 后忘记切回 Debug, 进而出现调试信息异常、断点行为不符合预期等问题。

```

add_executable(odt_callback_bench "speed
    ↪ test/odt_callback_bench.cpp")
target_compile_options(odt_callback_bench
    ↪ PRIVATE "SHELL:${CMAKE_CXX_FLAGS_RELEASE}")

```

1.5.3 小心 vector 大小等相关调试信息信息显示错误

```

#ifdef LOCAL
#define _GLIBCXX_DEBUG
#endif

```

不要在这个 LLDB 调试器下使用这个东西, 可能会导致显示错误, 因为类型变为了 std::__debug::vector, 调试器不支持。

1.6 本地手测时怎么发 EOF

本地跑 / 这类按 EOF 终止的 CP 代码, 手动在终端输完样例后要发 EOF 让 stdin 关流, 否则程序一直阻塞读不到末尾。**EOF 的发送键在不同系统/终端里不一样**:

- macOS / Linux 真终端: Ctrl-D (在空输入行上按)。
- Windows cmd / PowerShell: Ctrl-Z 后按 Enter。
- WSL / Git Bash on Windows: Ctrl-D (同 Unix)。
- CLion / VSCode Run 控制台: 看 Clion KeyMap SendEOF, mac/Linux ⇒ Cmd-D Ctrl-D, Windows ⇒ Ctrl-Z+Enter。

1.7 CLion 常用功能与快捷键（默认 keymap）

赛场 / 平时最常用的几个 CLion 操作与默认快捷键如下。赛场机一律 Windows / Linux，认第二列即可；第三列是 macOS 默认 keymap，平时在自己机器上对照用。**Run context configuration** 指「运行光标当前所在的那个目标」——多目标 CMake 工程里免去手动切下拉框，直接跑当前打开的这道题；**Reformat Code** 是一键格式化当前文件 / 选区；**Rename** 是语义级重命名，改变量 / 函数名时所有引用一起改，比全文替换安全（不会误伤字符串和同名的别的符号）。快捷键随 keymap 版本略有出入，忘了就用 **Find Action** (Ctrl+Shift+A) 搜功能名，弹出项右侧直接显示当前绑定。

Move Line 按物理行搬、不管语法；**Move Statement** 按语句 / 代码块整体对调，且拒绝挪到破坏语法的位置。复制行只有 **Duplicate Line**（只能向下复制一份），没有 VSCode 那种「上下移动时自动复制」。

功能	Windows / Linux	macOS
Run context configuration（跑光标所在目标）	Ctrl+Shift+F10	Ctrl+Shift+R
Run（当前选中的配置）	Shift+F10	Ctrl+R
Debug（当前选中的配置）	Shift+F9	Ctrl+D
Build Project（只编译不跑）	Ctrl+F9	Cmd+F9
Reformat Code（格式化当前文件 / 选区）	Ctrl+Alt+L	Opt+Cmd+L
Rename（重命名符号 + 所有引用）	Shift+F6	Shift+F6
注释 / 取消注释行	Ctrl+/ Ctrl+Y	Cmd+/ Cmd+Backspace
Duplicate Line（复制当前行 / 选区到下方）	Ctrl+D	Cmd+D
删除整行	Ctrl+Y	Cmd+Backspace
Move Line（纯按行上 / 下移）	Alt+Shift+Up/Down	Opt+Shift+Up/Down
Move Statement（按语句块上 / 下移）	Ctrl+Shift+Up/Down	Cmd+Shift+Up/Down
Find Action（搜任意功能 / 快捷键）	Ctrl+Shift+A	Cmd+Shift+A

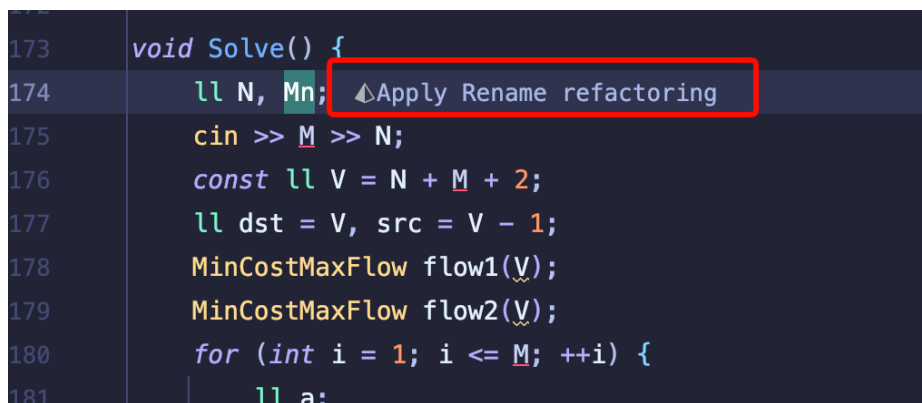
1.8 CLion 语言引擎：赛场机上先切 Nova

CLion 有两套语言引擎：**Classic**（补全 / 高亮走 clangd）和 **Nova**（换成 ReSharper C++ 引擎，核心功能不走 clangd）。**赛场机开机先切 Nova**——最直接的好处是类模板补全带尖括号：敲 `vec` 回车，Nova 给 `vector<>` 且光标落在尖括号中间，Classic 只给 `vector`。竞赛里 `vector` / `map` 满屏都是，省下的按键很可观。

Classic 这边没有开关能打开：clangd 只对函数模板补尖括号，类模板一律不补（C++17 有了 CTAD，`vector v{1, 2}`；本身合法，判断不出要不要那对尖括号）。

切换：右侧工具栏齿轮 → **Switch to Nova Engine** → **Enable and Restart**；或 **Settings** → **Advanced Settings** → 勾 **Use the ReSharper C++ language engine (CLion Nova)**。要重启 + 重新解析项目，所以热身赛就切好；设置每台机独立，换机器要重切。CLion 2025.3 起 Nova 已是默认，新版本不用管这节。

切到 Nova 后还多一个改名捷径：直接把变量名敲成新的，行内浮出 **Apply Rename refactoring**（下图），点它或按 **Alt+Enter** 回车，所有引用一起改掉，省掉先按 **Shift+F6** 开对话框这一步。Classic 引擎没有这个提示，行内提示样式则要 CLion 2025.2 以上。



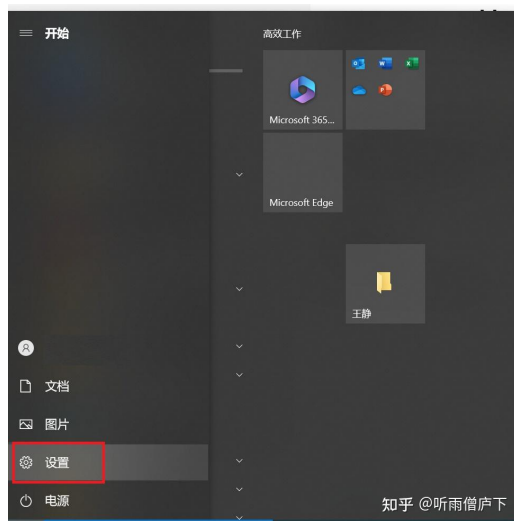
```
173 void Solve() {
174     ll N, M;
175     cin >> M >> N;
176     const ll V = N + M + 2;
177     ll dst = V, src = V - 1;
178     MinCostMaxFlow flow1(V);
179     MinCostMaxFlow flow2(V);
180     for (int i = 1; i <= M; ++i) {
181         ll a;
```

1.9 Windows：禁用 Shift 切换中英文输入

Windows 赛场机上敲代码，误碰 **Shift**（打大写、打符号松手稍慢）就会被输入法切到中文模式，接着敲出一串全角字符——开赛前先把这个切换键关掉。

1.9.1 微软拼音（系统自带，Win10 逐步截图）

1. 开始菜单 → 设置：



2. 进入时间和语言：



3. 左侧栏切到语言：



4. 「首选语言」列表里点中文 (简体，中国)：



知乎 @听雨僧庐下

5. 点展开出来的选项按钮：



知乎 @听雨僧庐下

6. 键盘列表里点微软拼音 → 进入按键：



知乎 @听雨僧庐下

7. 「中/英文模式切换」里把 Shift 的勾去掉（保留 Ctrl + 空格键，这就是之后手动切中英文的键）：



知乎 @听雨僧庐下

改完后 Shift 只做普通修饰键；切中英文用 Ctrl + Space，切输入法 / 语言用 Win + Space。Win11 路径名稍有不同：设置 → 时间和语言 → **语言和区域** → 中文右侧「…」 → 语言选项 → 微软拼音 → 键盘选项 → 按键，最后一步相同。

1.9.2 搜狗输入法

1. 任务栏搜狗图标打开工具面板 → **更多设置 (P)**：



2. 属性设置 → **按键** → 「中英切换」选无（不想彻底禁可选 Ctrl）：



QQ / 百度输入法路径类似，都在「按键设置」里。

更省事的做法（比赛全程不需要中文输入时）：直接在上面第 4 步的语言列表里删掉中文输入法或把默认语言切到 ENG，Shift 就完全没有切换语义了。快速定位入口：开始菜单搜「输入法」→「编辑语言和键盘选项」。

2 基础数学

2.1 快速幂

```
// 在  $k \leq 0$  的时候返回 1
ll binpow(ll a, ll k, const ll &p=mod){
    ll res=1;
    a%=p;
    while(k>0){
        if((k&1)==1) res=a*res%p;
        a=a*a%p;
        k>>=1;
    }
    return res;
}
```

2.2 矩阵快速幂

注意，我们这个东西依赖于后面的这个**自动取模类**，所以代码还是比较长的。

```
/*
 * 2025/11/5
 * → https://www.luogu.com.cn/record/245403611
 * → 矩阵快速幂模版题
 * 2025/11/5
 * → https://www.luogu.com.cn/record/245411884
 * → P1939 矩阵加速（数列）
 * 2026/3/4 去除了过时的宏定义和没有意义的压行
 */
template<class T>
struct Matrix {
    int n, m;
    vector<vector<T>> > a; // 1-based

    Matrix() : n(0), m(0) {}

    Matrix(int n_, int m_, const T &val = T()) {
        assign(n_, m_, val);
    }
};
```

```
}

// 用 1-based 的 vv 构造
Matrix(const vector<vector<T>> &vv) {
    load(vv); }

// vv.size() == n+1, vv[1].size() == m+1
void load(const vector<vector<T>> &vv) {
    n = (int) vv.size() - 1;
    m = n ? (int) vv[1].size() - 1 : 0;
    a = vv;
}

void assign(int n_, int m_, const T &val =
    T()) {
    n = n_;
    m = m_;
    a.assign(n + 1, vector<T>(m + 1, val));
}

// 重载
vector<T> &operator[](int i) { return a[i]; }
const vector<T> &operator[](int i) const {
    return a[i]; }

// += 负责真正干活
Matrix &operator+=(const Matrix &rhs) {
    assert(n == rhs.n && m == rhs.m);
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            a[i][j] += rhs[i][j];
        }
    }

    return *this;
}

// + 基于 +=
Matrix operator+(const Matrix &rhs) const {
```

```

Matrix res(*this);
res += rhs;
return res;
}

Matrix &operator--(const Matrix &rhs) {
    assert(n == rhs.n && m == rhs.m);
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            a[i][j] -= rhs[i][j];
        }
    }

    return *this;
}

Matrix operator-(const Matrix &rhs) const {
    Matrix res(*this);
    res -= rhs;
    return res;
}

Matrix &operator+=(const Matrix &rhs) {
    *this = (*this) * rhs;
    return *this;
}

Matrix operator*(const Matrix &rhs) const {
    assert(m == rhs.n);
    Matrix res(n, rhs.m, T());
    for (int i = 1; i <= n; ++i) {
        for (int k = 1; k <= m; ++k) {
            if (a[i][k] != T()) {
                for (int j = 1; j <= rhs.m; ++j) {
                    res[i][j] += a[i][k] * rhs[k][j];
                }
            }
        }
    }
    return res;
}

static Matrix identity(int n) {
    Matrix I(n, n, T());
    for (int i = 1; i <= n; ++i) I[i][i] = T(1);
    return I;
}

friend ostream &operator<<(ostream &os, const
↪ Matrix &ma) {
    for (int i = 1; i <= ma.n; ++i) {
        for (int j = 1; j <= ma.m; ++j) {
            os << ma[i][j] << " ";
        }
        os << "\n";
    }
    return os;
}
};

// 矩阵快速幂：必须是方阵
template<class T>
Matrix<T> mat_pow(Matrix<T> base, long long
↪ exp) {

```

```

    assert(base.n == base.m);
    Matrix<T> res = Matrix<T>::identity(base.n);
    while (exp > 0) {
        if (exp & 1) res = res * base;
        base = base * base;
        exp >>= 1;
    }
    return res;
}

```

2.2.1 矩阵快速幂加速递推

已知一个数列 a ，它满足：

$$a_x = \begin{cases} 1 & x \in \{1, 2, 3\} \\ a_{x-1} + a_{x-3} & x \geq 4 \end{cases}$$

求 a 数列的第 n 项对 $10^9 + 7$ 取余的值。

对于递推式 $a_x = a_{x-1} + a_{x-3}$ ，因为递推涉及到了前 3 项的状态，我们可以构造一个大小为 3×1 的状态

列向量 $\begin{bmatrix} a_x \\ a_{x-1} \\ a_{x-2} \end{bmatrix}$ 。

根据递推关系，我们可以将其展开为矩阵乘法的形式，寻找转移矩阵 T ：

$$\begin{aligned} a_x &= 1 \cdot a_{x-1} + 0 \cdot a_{x-2} + 1 \cdot a_{x-3} \\ a_{x-1} &= 1 \cdot a_{x-1} + 0 \cdot a_{x-2} + 0 \cdot a_{x-3} \\ a_{x-2} &= 0 \cdot a_{x-1} + 1 \cdot a_{x-2} + 0 \cdot a_{x-3} \end{aligned}$$

通过提取系数，可以得出转移矩阵 T ：

$$\begin{bmatrix} a_x \\ a_{x-1} \\ a_{x-2} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} a_{x-1} \\ a_{x-2} \\ a_{x-3} \end{bmatrix}$$

(这里就是普通的矩阵乘法，按照矩阵乘法规则展开，上式和下式是等价的)

初始状态列向量为 $V = \begin{bmatrix} a_3 \\ a_2 \\ a_1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$ 。

将转移矩阵 T 自乘 $n-1$ 次后，再乘以初始向量 V ，即可得到最终状态：

$$T^{n-1}V = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}^{n-1} \begin{bmatrix} a_3 \\ a_2 \\ a_1 \end{bmatrix} = \begin{bmatrix} a_{n+2} \\ a_{n+1} \\ a_n \end{bmatrix}$$

由此可见，经过 $n-1$ 次状态转移后，底部的元素恰好从 a_1 递推变为了 a_n 。因此结果矩阵的第 3 行第 1 列即为所求的 a_n （在代码中由于加入了空的 $\{\}$ 占位使下标从 1 开始，对应为 `mat[3][1]`）。

```

void Solve() {
    ll n;
    cin >> n;
    vector<vector<modi>> A{
        {}, // 递推矩阵
    }
}

```



```

    {{}, 1, 0, 1}, // fn = fn-1 + 0*fn-2 +
    ↪ fn-3
    {{}, 1, 0, 0},
    {{}, 0, 1, 0}
},
B = {          // 原向量 v
    ↪ (也可以认为是一个矩阵)
    {{},
    {{}, 1}, // a3
    {{}, 1}, // a2
    {{}, 1}  // a1
};
Matrix mat(A), matb(B);
mat = mat_pow(mat, n-1);
mat*=matb; // 递推矩阵 T* 原向量 v
// a1 递推 n-1 次, 是这个 an
cout << mat[3][1] << "\n";
}

```

对于递推式 $S_n = 22S_{n-1} - S_{n-2}$, 我们可以将其转化为矩阵乘法的形式来加速计算。

首先构造状态列向量 $\begin{bmatrix} S_n \\ S_{n-1} \end{bmatrix}$, 根据递推关系可以得出转移矩阵 T :

$$\begin{bmatrix} S_n \\ S_{n-1} \end{bmatrix} = \begin{bmatrix} 22 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} S_{n-1} \\ S_{n-2} \end{bmatrix}$$

初始状态列向量为 $V = \begin{bmatrix} S_2 \\ S_1 \end{bmatrix} = \begin{bmatrix} 482 \\ 22 \end{bmatrix}$ 。

将转移矩阵 T 自乘 $n-1$ 次后, 再乘以初始向量 V , 即可得到最终状态:

$$T^{n-1}V = \begin{bmatrix} 22 & -1 \\ 1 & 0 \end{bmatrix}^{n-1} \begin{bmatrix} S_2 \\ S_1 \end{bmatrix} = \begin{bmatrix} S_{n+1} \\ S_n \end{bmatrix}$$

由此可见, 计算结果矩阵的第 2 行第 1 列即为所求的 S_n 。代码中因为使用了一个空的 `{}` 进行占位 (即下标从 1 开始), 所以对应的取值为 `T[2][1]`。

```

vector<vector<modi>> >
// 构造转移矩阵 T
// [ S_n ] = [ 22 -1 ] * [ S_{n-1} ]
// [ S_{n-1} ] [ 1 0 ] [ S_{n-2} ]
// 注意: 用 {} 占位使得二维 vector 的下标从 1 开始
tmp={{},
    {{},22,-1},
    {{},1,0},
    {}
};
Matrix T(tmp);

// 将转移矩阵自乘 n-1 次
T=mat_pow(T,n-1);

// 构造初始状态列向量 V
// V = [ S_2 ] = [ 482 ]
//     [ S_1 ] [ 22 ]
vector<vector<modi>> tmp2=
    {{},
    {{},482},
    {{},22},
    {}
};

```

```

Matrix V(tmp2);

// 将转移矩阵和初始向量相乘: T^{n-1} * V
T*=V;

// 提取结果 S_n
// 经过 n-1 次转移后, 结果列向量变为 [ S_{n+1} ]
//                                     [ S_n ]
// 故所求结果在第 2 行第 1 列 (1-indexed)
modi res=T[2][1];

```

2.3 自动取模类

```

// T: 底层存储类型 (uint32_t / uint64_t)
// MOD: 模数
// Wide: 乘法时用的更宽类型
template <class T, T MOD, class Wide = ull>
struct modint {
    T v;
    modint(ll x = 0) {
        ll t = x % (ll)MOD;
        if (t < 0) t += MOD;
        v = (T)t;
    }

    T val() const { return v; }

    modint& operator+=(const modint &o) {
        T x = v + o.v;
        if (x >= MOD) x -= MOD;
        v = x;
        return *this;
    }

    modint& operator-=(const modint &o) {
        T x = v >= o.v ? v - o.v : (T)(v + MOD -
        ↪ o.v);
        v = x;
        return *this;
    }

    modint& operator*=(const modint &o) {
        Wide t = (Wide)v * (Wide)o.v;
        v = (T)(t % MOD);
        return *this;
    }

    // 幂
    static modint powmod(modint a, ll e) {
        modint r = 1;
        while (e) {
            if (e & 1) r *= a;
            a *= a;
            e >>= 1;
        }
        return r;
    }

    // 逆元 (假定 MOD 是质数)
    static modint inv(modint a) {
        return powmod(a, (ll)MOD - 2);
    }
}

```



```

modint& operator/=(const modint &o) {
    return *this *= inv(o);
}

// 一元
modint operator+() const { return *this; }
modint operator-() const { return modint(0) -
    ↪ *this; }

// 二元
friend modint operator+(modint a, const
    ↪ modint &b) { return a += b; }
friend modint operator-(modint a, const
    ↪ modint &b) { return a -= b; }
friend modint operator*(modint a, const
    ↪ modint &b) { return a *= b; }
friend modint operator/(modint a, const
    ↪ modint &b) { return a /= b; }

friend bool operator==(const modint &a, const
    ↪ modint &b) { return a.v == b.v; }
friend bool operator!=(const modint &a, const
    ↪ modint &b) { return a.v != b.v; }

friend ostream& operator<<(ostream &os, const
    ↪ modint &x) {
    return os << x.v;
}
friend istream& operator>>(istream &is,
    ↪ modint &x) {
    ll t;
    if (!(is >> t)) return is;
    x = modint(t);
    return is;
}
};
using modi=modint<uint32_t,mod>;

```

2.4 异或线性基 (linear basis)

异或线性基用于解决在一个整数集合中，任意选取元素进行异或操作所能产生的数值组合问题。其核心原理基于线性代数中的向量空间，将整数的二进制表示看作位于 $\mathbb{F}_2$ 域（即模 2 加法，等价于异或）上的向量。

模板中各核心操作的理论基础如下：

- **插入 (insert)**：本质是向极大线性无关组中添加新向量的过程。从高位到低位遍历，若当前数字的第 i 位为 1，且该位已存在基底 d_i ，则将数字异或 d_i 以消去该位；若不存在基底，则将当前数字作为 d_i 存入，插入成功。若最终数字变为 0，说明它可由已有基底线性表出。
- **查询最大值 (query_max)**：基于贪心思想。从高位到低位遍历，对于第 i 位的基底 d_i ，因为它是集合中唯一最高位为 i 的基底，若当前答案异或 d_i 后能变大，则必定选择异或。因为无论后面低位如何选择，都无法弥补当前第 i 位缺失的 1。
- **查询最小值 (query_min)**：如果在插入时出现过能被线性表出的数字（即能异或出 0），那么最小

值就是 0；否则，由于基底中的元素互相无法表出对方，线性基数组中最小的非零基底就是能异或出的最小值。

- **重构与求第 k 小 (rebuild & query_kth)**：rebuild 的本质是对线性基矩阵进行高斯消元，化为“简化阶梯型矩阵”。经过重构，对于任意基底 p_j ，其所有的二进制位 1 在其它基底的对应位上一定都是 0。这使得基底之间变成了完全独立的“正交”状态，它们能表出的所有异或和严格按基底的选择与否呈现字典序递增。因此，将排名 k 转化为二进制表示，若第 j 位为 1，则异或上第 j 小的非零基底，即可拼凑出第 k 小的异或和。

```

/*
 * 2026-04-06: AC
 * ↪ https://vjudge.net/solution/69022982
 * 主要是测试了 rebuild 和 query_kth
 * 2026-04-06: AC
 * ↪ https://www.luogu.com.cn/record/272234814
 * 主要测试了 merge 功能和这个 query_max 功能
 * 2026-04-07: AC https://acm.hdu.edu.cn/contes
 * ↪ t/view-code?cid=1202&rid=3312&from=rank
 * 测试了 query_min_with
 */
struct LinearBasis {
    static const int MAXL = 60;
    vector<long long> d;
    vector<long long> p; // 正交化后的基底，用于求第
    ↪ k 小
    int cnt; // 线性基内元素的数量 (秩)
    bool zero; // 原数组是否能异或出 0

    LinearBasis() {
        d.assign(MAXL + 1, 0);
        cnt = 0;
        zero = false;
    }

    // 向极大线性无关组中添加新向量
    bool insert(long long val) {
        // 从高到低遍历二进制位
        for (int i = MAXL; i >= 0; i--) {
            // 若当前数字的第 i 位为 1
            if (val & (1LL << i)) {
                // 若不存在该位的基底，则作为新基底存入，
                ↪ 插入成功
                if (!d[i]) {
                    d[i] = val;
                    cnt++;
                    return true;
                }
                // 若已存在该位基底，则异或消去该位的 1
                val ^= d[i];
            }
        }
        // 若最终数字变为 0，说明它可由已有基底线性表出
        zero = true;
        return false;
    }

    // 查询能异或出的最大值

```

```

long long query_max() {
    long long res = 0;
    // 基于最高位独占特性的贪心，从高位到低位遍历
    for (int i = MAXL; i >= 0; i--) {
        // 若当前答案异或基底 d[i] 后能变大，
        // 则必然选择异或
        if ((res ^ d[i]) > res) {
            res ^= d[i];
        }
    }
    return res;
}

// 查询能异或出的最小值
long long query_min() {
    // 若在插入时出现过能被线性表出的数字
    // (即能异或出 0)
    if (zero) return 0;
    // 否则，线性基数组中最小的非零基底就是能异或
    // 出的最小值
    for (int i = 0; i <= MAXL; i++) {
        if (d[i]) return d[i];
    }
    return 0;
}

// 重构线性基，通过类似高斯消元化为简化阶梯型矩阵
void rebuild() {
    // 使所有基底互不干扰，达到完全正交状态
    for (int i = MAXL; i >= 0; i--) {
        for (int j = i - 1; j >= 0; j--) {
            // 如果 d[i] 的第 j 位为 1，则用 d[j]
            // 消去这一位
            if (d[i] & (1LL << j)) {
                d[i] ^= d[j];
            }
        }
    }
    p.clear();
    // 将正交化后的非零基底提取出来，
    // 方便后续按字典序查询
    for (int i = 0; i <= MAXL; i++) {
        if (d[i]) p.push_back(d[i]);
    }
}

// 查询去重后的第 k 小值 (要求先调用 rebuild)
long long query_kth(long long k) {
    // 如果原集合能异或出 0，并且 0 算第一小，
    // 先扣除 0 的排名
    if (zero) k--;
    if (k == 0) return 0;
    // 若查询排名超过了能异或出的总个数 (2^cnt - 1)
    // 一个非零值)
    if (k >= (1LL << cnt)) return -1;

    long long res = 0;
    // 将排名 k 转化为二进制表示
    for (int i = 0; i < (int) p.size(); i++) {
        // 若第 i 位为 1，则异或上正交化后的第 i
        // 小的非零基底
        if (k & (1LL << i)) res ^= p[i];
    }
}

```

```

return res;
}

// 合并另一个线性基
void merge(const LinearBasis &other) {
    for (int i = 0; i <= MAXL; i++) {
        // 将另一个线性基的非零基底全部插入当前线性
        // 基
        if (other.d[i]) insert(other.d[i]);
    }
    zero |= other.zero;
}

// 给定初值 init，从线性基里选任意子集 XOR，使
// init ^ subset_xor 最小
// 典型场景：1~n 最短 XOR 路径 (HDU「月之都」) —
// 先求任意一条路径的异或和
// init = (1 ~ n) ^ 额外边的 x^y^w，再用环
// (linearly 入基的 x^y^w) 去最小化
long long query_min_with(long long init) {
    long long res = init;
    // 从高位到低位贪心：d[i] 的最高位就是第 i 位，
    // 不会污染更高位；
    // 若 res 的第 i 位为 1，异或 d[i] 能把它消掉，
    // res 严格变小；反之不改
    for (int i = MAXL; i >= 0; i--) {
        if ((res ^ d[i]) < res)
            res ^= d[i];
    }
    return res;
}
}

```

2.5 带权 bitset 线性基 (F2 线性方程组求解)

给出 K 个已知向量 $\text{mask}_j \in \mathbb{F}_2^N$ 及各自挂着的权值 S_j ，询问目标向量能否被它们线性表出：能表出就返回对应权值的异或，表不出返回 -1 。本质是对增广矩阵 $[\text{mask} | S]$ 做高斯消元——左半边每做一次行异或，右半边同步做一次； N 远大于 64 时整行用 bitset 异或。来源：HDU 2026 多校 5「链上 Nim」。

- $d[i]$ 非空时最高位恰为第 i 位；query 把目标消成全 0 即表出成功，还剩非零位说明不在张成的子空间里，返回 -1 。
- 落位后必须立刻 break (代码中高亮那行)： $d[i]$ 存的是 mask 的副本，不 break 会带着同一个 mask 继续往低位走、在更低的空位上再落位一次，秩虚增后 query 会把本来表不出的 mask 判成可表出。
- 单次 insert / query 为 $O(N^2/w)$ ，整体 $O((K+Q)N^2/w)$ ，空间 $O(N^2/w)$ 。

行	mask (位 5 → 0)						val
d_5	1	0	1	0	1	0	7
d_3	0	0	1	0	0	1	4

d_4, d_2, d_1, d_0 仍为空 (none())

图注: $N = 6$, 依次插入 $(\{5, 3, 1\}, 7)$ 、 $(\{3, 0\}, 4)$ 、 $(\{5, 1, 0\}, 3)$ 。
 前两个各自落位成主元行 (蓝底 = 主元位, 也就是该行最高位);
 第三个先异或 d_5 变成 $\{3, 0\}$ 、权值 $3 \oplus 7 = 4$, 再异或 d_3 归零,
 说明它能被前两行表出, 直接丢弃。查询 $\{5, 1, 0\}$ 消到全 0, 返回
 $7 \oplus 4 = 3$; 查询 $\{5, 0\}$ 消到最后剩第 1 位而 d_1 为空, 返回 -1 。

```
/*
 * 2026-08-04: AC https://acm.hdu.edu.cn/contes
 * t/view-code?cid=1233&rid=5589
 * HDU 2026 多校 5 1010 「链上 Nim」: 给出  $K$  组
 *   (位置集合  $mask$ , 该集合对应的值  $S$ ),
 *  $Q$  次询问某个位置集合的值 -- 即在  $F_2$  上把询问
 *    $mask$  用已知  $mask$  线性表出,
 * 答案为对应  $S$  的异或; 表不出来返回  $-1$ 。
 * 与普通异或线性基的差别: 主元向量是  $bitset<N>$  ( $N$ 
 *   可远大于  $64$ ), 且每行额外挂一个权值  $val$ 。
 */
template<int N>
struct LinearBasis {
    bitset<N> d[N]; // d[i]: 以第  $i$  位为主元的行
    // (已消去所有更高位)
    ll val[N]; // 该行对应的异或值

    void clear() {
        for (int i = 0; i < N; ++i) {
            d[i].reset();
            val[i] = 0;
        }
    }

    void insert(bitset<N> mask, ll v) {
        for (int i = N - 1; i >= 0; --i) {
            if (mask[i]) {
                if (d[i].none()) {
                    d[i] = mask;
                    val[i] = v;
                    break; // [KEY] 落位即停: mask 连同 v
                        // 成为第  $i$  位的主元行
                } else {
                    mask ^= d[i];
                    v ^= val[i];
                }
            }
        }
        // 走完循环没 break: mask 被消成 0,
        // 说明它已被已有行表出, 连同 v 一起丢弃
    }

    ll query(bitset<N> mask) {
        ll res = 0;
        for (int i = N - 1; i >= 0; --i) {
            if (mask[i]) {
                if (d[i].any()) {
                    mask ^= d[i];
                    res ^= val[i];
                }
            }
        }
        if (mask.any()) {
            return -1; // 消不干净: mask
            // 不在已知集合张成的子空间里
        }
        return res;
    }
};
```

```
}
};
```

2.6 区间异或线性基 (prefix linear basis)

处理「区间内任取若干元素异或最大值」的问题。朴素做法是对每个区间重建线性基 ($O(n \log V)$ 单次), 但不能在线。下面这套前缀线性基 + 贪心保留下标靠右元素的写法可以做到 $O(\log V)$ 单次查询, 且天然支持「末尾追加」操作 (强制在线场景)。

模板中各核心操作的理论基础如下:

- **追加 (append):** 对每个前缀 $[1..i]$ 维护一个线性基 $L[i]$, 由 $L[i-1]$ 增量得到。插入新元素 (val, idx) 时, 从高位到低位走, 对每一个 val 仍存在的位 i : 若 d_i 为空, 直接落位; 否则比较两者下标, 将下标更大的那一对存入 d_i 、 pos_i , 把下标更小的那一对继续往低位消。这样 $L[r]$ 的每条基底都尽可能用下标靠右的元素来表达。
- **区间查询 (query_max):** 直接取 $L[r]$, 从高位到低位贪心, 但只接受 $pos_i \geq l$ 的基底。正确性来源于「靠右优先」的不变量: $L[r]$ 中所有 $pos_i \geq l$ 的基底, 恰好张成 $a[l..r]$ 的异或子空间 (每条基底都可写成只用 $a[l..r]$ 内元素的某个异或组合)。
- **空间:** 每个 $L[i]$ 是 BITS 位的基底数组 (再加一份 pos 数组), 整体 $O(n \cdot \text{BITS})$ 。值域 $\leq 2^{30}$ 时单层 240 字节, $n + m = 10^6$ 约 240 MB; 若值域更小可压成 `uint16_t` 进一步节省。

```
/*
 * 2026-04-29: AC
 * https://vjudge.net/solution/69555764
 * 区间异或线性基模板题: HDU 6579 Operation
 * op 0 l r: 区间  $[l, r]$  内子集异或最大值
 *   (强制在线)
 * op 1 x : 在数组末尾追加  $x$  (强制在线)
 */
struct PrefixLinearBasis {
    static constexpr int BITS = 30;
    using T = unsigned;

    struct Layer {
        T d [BITS] = {}; // d[i]: 最高位为  $i$ 
        // 的基底
        int pos[BITS] = {}; // pos[i]: 当前  $d[i]$ 
        // 对应的元素下标 (1-indexed)
    };

    vector<Layer> L; // L[k]: a[1..k]
    // 的前缀线性基

    PrefixLinearBasis() {
        L.emplace_back(); // L[0] 为空基
    }
    void reserve(int n) { L.reserve(n + 1); }
    void clear() { L.clear(); }
    void insert(T x, int pos) {
        L.emplace_back();
        int i = L.size() - 1;
        while (i >= 0) {
            if (x >= (1LL << i)) {
                if (!L[i].d[i]) {
                    L[i].d[i] = x;
                    L[i].pos[i] = pos;
                    break;
                }
                x ^= L[i].d[i];
            }
            i--;
        }
    }
    int query(int l, int r) {
        Layer &Lr = L[r];
        int res = 0;
        for (int i = BITS - 1; i >= 0; --i) {
            if (Lr.d[i] & (res ^ Lr.d[i]) > res) {
                if (Lr.pos[i] >= l) {
                    res ^= Lr.d[i];
                }
            }
        }
        return res;
    }
};
```

```

// 末尾追加 val, 自动成为 a[size()+1]
void append(T val) {
    Layer cur = L.back();
    int idx = (int) L.size();
    for (int i = BITS - 1; i >= 0; --i) {
        if (!(val >> i) & 1) continue;
        if (!cur.d[i]) { //
            ↪ 该位空, 落位即可
            cur.d[i] = val;
            cur.pos[i] = idx;
            break;
        }
        // 让 d[i] 始终保留下标更靠右的那一对
        if (cur.pos[i] < idx) {
            swap(cur.d[i], val);
            swap(cur.pos[i], idx);
        }
        val ^= cur.d[i];
    }
    L.push_back(cur);
}

// a[l..r] 子集异或最大值 (1-indexed, l <= r)
T query_max(int l, int r) const {
    T ans = 0;
    const Layer &b = L[r];
    for (int i = BITS - 1; i >= 0; --i) {
        if (b.pos[i] < l) continue; //
        ↪ 该基底用到了 l 之前的元素, 跳过
        if ((ans ^ b.d[i]) > ans) ans ^= b.d[i];
    }
    return ans;
}
};

```

2.7 n 进制异或线性基

n 进制异或线性基 (模 p 域上的线性基) 用于处理在 $\mathbb{F}_p$ (其中 p 为质数) 域上的向量空间问题。与普通的 $\mathbb{F}_2$ 域 (二进制异或) 类似, 在 $\mathbb{F}_p$ 域上, 每个数都可以分解为 p 进制的各位数字, 构成一个向量。数字间的“ p 进制不进位加法”等价于向量的逐元模 p 加法。

模板中各核心操作的理论基础如下:

- **转化与消元 (insert & query):** 首先将数字转化为 p 进制向量。利用类似高斯消元法从高到低进行处理, 对于每一位, 若当前位非零:
 - **插入时:** 若该位不存在基底, 说明找到了一个新的线性无关向量。为了方便后续消去其他向量的该位, 乘以当前最高位元素的乘法逆元, 将最高位“归一化”为 1 后存入矩阵 (mat), 插入成功。若已存在基底, 则减去当前位数字与对应基底的乘积, 从而消去当前位的非零值。
 - **查询时:** 逻辑类似。若消元过程中遇到当前位非零但对位无基底, 说明包含无法由当前基表出的独立分量, 返回不可表出; 若最终整个向量被消为全 0, 则说明该数字可由基底完全线性表出。

```

/*
 * AC
 * https://acm.hdu.edu.cn/contest/view-code?cid=
 ↪ =1199&rid=7820&from=rank
 * n 进制异或线性基 (模 p 域)
 */
class modn_XOR_base {
private:
    vector<ll> base; // base[i] 记录最高位为 i
    ↪ 的那个原始数字
    vector<vector<ll>> mat; // mat[i] 记录最高位为
    ↪ i 的归一化后的 p 进制向量
    ll mod; // 模数 p (必须为质数)

    // 快速幂用于求逆元
    ll power(ll a, ll b) const {
        ll res = 1;
        a %= mod;
        while (b > 0) {
            if (b & 1) res = (res * a) % mod;
            a = (a * a) % mod;
            b >>= 1;
        }
        return res;
    }

    // 费马小定理求乘法逆元 (mod 必须为质数)
    ll modInverse(ll n) const {
        return power(n, mod - 2);
    }

public:
    modn_XOR_base(ll Mod) {
        mod = Mod;
        base.assign(64, 0);
        mat.assign(64, vector<ll>(64, 0));
    }

    // 向极大线性无关组中添加新向量 (p 进制数字)
    bool insert(ll x) {
        ll orig_x = x;
        vector<ll> v(64, 0);

        // 1. 将数字转化为 p 进制向量 (低位到高位)
        for (int i = 0; i < 64; ++i) {
            v[i] = x % mod;
            x /= mod;
        }

        // 2. 从高位到低位遍历, 进行高斯消元
        for (int i = 63; i >= 0; --i) {
            if (v[i] == 0) continue; // 当前位为 0,
            ↪ 不需要消元

            // 若不存在该位的基底, 则作为新基底存入
            if (base[i] == 0) {
                // 求出最高位数字的逆元,
                ↪ 用于将最高位归一化为 1
                ll inv = modInverse(v[i]);
                // 整个向量乘以逆元, 存入 mat[i]
                for (int j = 0; j <= i; ++j) {
                    mat[i][j] = (v[j] * inv) % mod;
                }
            }
        }
    }

```

/*

```

    base[i] = orig_x; // 记录原始数字
    return true;
}

// 若已存在该位基底,
// ↪ 利用对应基底消去当前位的非零值
ll k = v[i];
for (int j = 0; j <= i; ++j) {
    // 减去基底对应位的 k 倍 (k
    // ↪ 刚好是当前最高位的值,
    // ↪ 因为基底最高位已被归一化为 1)
    v[j] = (v[j] - k * mat[i][j]) % mod;
    if (v[j] < 0) v[j] += mod; //
    // ↪ 保证结果为正
}
}
// 若最终向量变为全 0, 说明可由已有基底线性表出
return false;
}

// 判断数字 x 能否由当前线性基表出 (即 x
// ↪ 是否在张成空间内)
bool query(ll x) const {
    vector<ll> v(64, 0);

    // 将数字转化为 p 进制向量
    for (int i = 0; i < 64; ++i) {
        v[i] = x % mod;
        x /= mod;
    }

    // 从高位到低位进行消元
    for (int i = 63; i >= 0; --i) {
        if (v[i] == 0) continue;

        // 若当前位不为 0, 但基底中对应位为空,
        // ↪ 说明带有新的独立分量, 无法被表出
        if (base[i] == 0) return false;

        // 若存在基底, 则继续消元
        ll k = v[i];
        for (int j = 0; j <= i; ++j) {
            v[j] = (v[j] - k * mat[i][j]) % mod;
            if (v[j] < 0) v[j] += mod;
        }
    }

    // 整个向量被成功消为全 0, 说明可被线性表出
    return true;
}
};

```

2.8 MEX (Minimum Excluded Value)

MEX (Minimum Excluded Value) 通常指一个非负整数集合中未出现的最小非负整数。

2.9 中位数 (Median)

本节约定 **长度奇数** N 的数组, 中位数即排序后的第 $\lceil N/2 \rceil$ 个元素 (也等于第 $(N+1)/2$ 个)。中位数题的“特征气味”: 题面里提到 **排序后取中间值 / 划分若干**

段、每段中位数相同 / 操作不改变中位数之类。

2.9.1 中位数的双向判定不等式

“ m 是数组 A 的中位数”的等价条件是**两条对称的下界同时成立**:

$$\#\{a \leq m\} \geq \lceil N/2 \rceil \quad \text{且} \quad \#\{a \geq m\} \geq \lceil N/2 \rceil$$

为什么是“ $\leq$ ”和“ $\geq$ ”，而不是“ $<$ ”和“ $>$ ”: 要让 m 这个具体的值出现在中间位置, 必须同时有 $\geq \lceil N/2 \rceil$ 个不超过它的、 $\geq \lceil N/2 \rceil$ 个不少于它的——两条等号都包含 m 自己, 正是它落在中间的根据。如果换成严格不等式, 只能限定中位数 $\geq m$ 或 $\leq m$ 单边, 得不到 $= m$ 。

推论: 包含性自动达成。 两条相加, 等号情形给出

$$\#\{a \leq m\} + \#\{a \geq m\} = N + \#\{a = m\},$$

右边减 N 等于左边减 $N \geq 2\lceil N/2 \rceil - N = 1$ (奇 N), 所以 $\#\{a = m\} \geq 1$ 。即“ m 在数组里出现至少一次”是这两条下界的推论, 不是独立条件。判区间中位数等于 m 时不必单独判 m 在不在, 套这两条不等式即可——这是常踩的坑。

2.9.2 ± 1 编码 + 前缀和: $O(1)$ 区间中位数判定

要 $O(1)$ 判 “ $\text{med}(a[l..r]) = m$ ” (区间长度奇), 定义两套 ± 1 编码:

$$b_1[k] = \begin{cases} +1, & a_k \leq m \\ -1, & a_k > m \end{cases}, \quad b_2[k] = \begin{cases} +1, & a_k \geq m \\ -1, & a_k < m \end{cases}$$

分别取前缀和 B_1, B_2 。对于奇长 $\text{len} = r - l + 1$:

$$\begin{aligned} B_1[r] - B_1[l-1] &= \#\{a \leq m\} - \#\{a > m\} \\ &= 2\#\{a \leq m\} - \text{len}. \end{aligned}$$

所以 $\#\{a \leq m\} \geq \lceil \text{len}/2 \rceil = (\text{len} + 1)/2$ 当且仅当 B_1 差值 ≥ 1 。 B_2 同理。综合:

$$\begin{aligned} \text{med}(a[l..r]) = m &\iff B_1[r] - B_1[l-1] \geq 1 \\ &\quad \text{且 } B_2[r] - B_2[l-1] \geq 1 \end{aligned}$$

预处理 $O(N)$, 单次判 $O(1)$ 。这个 ± 1 + 双前缀和的组合是中位数题的**瑞士军刀**: 除了等于判定, 还能拓展到“区间中位数 $\geq m$ 个数” (只要一条 B_1 差 ≥ 1) 和“区间中位数 $\leq m$ 个数” (只要一条 B_2 差 ≥ 1)。

2.9.3 多段奇长度划分公共中位数 = 全局中位数

命题: 长度奇 N 的数组 a 被划分成 p 段奇长度子数组, 且**每段中位数都等于公共值 m** , 则 m 必等于 a 的整体中位数。

一行证明：每段 S_i 长 ℓ_i (奇)、中位数 m ，由判定不等式 $\#_{S_i}(\leq m) \geq (\ell_i + 1)/2$ 。对所有 p 段相加：

$$\#_a(\leq m) \geq \sum_{i=1}^p \frac{\ell_i + 1}{2} = \frac{N + p}{2} \geq \lceil N/2 \rceil.$$

$\#_a(\geq m)$ 同理。两条都成立 $\Rightarrow m$ 是 a 的中位数；奇 N 中位数唯一，所以 m 即整体中位数。推论：处理这类划分题时不必枚举公共中位数候选——直接 $O(N \log N)$ 排序取出 $a_{\lceil N/2 \rceil}$ 当 m 即可，问题立刻退化成“已知 m ，最大化奇长度段数”。

实战范例 (CF 1094C Median Partition)：把奇长 N 数组划分成最多奇长度子数组、每段中位数相同。先求出全局中位数 m (上一条命题)，再做 $O(N^2)$ DP: $f[i] = \max\{f[j] + 1 : 0 \leq j < i, (i - j) \text{ 奇}, \text{med}(a[j+1..i]) = m\}$ ，区间中位数判定走 ± 1 双前缀和 (上一条工具)。两件套合用， $\sum N^2 \leq 5000^2$ 直接通过。这道题的“难点”完全集中在“公共中位数 = 全局中位数”和“ ± 1 双前缀和”这两个性质上——把性质内化进板子后，类似题型可以“看到中位数 = 公共值”就直接套出 DP 框架，不再需要赛时重新发明判定。

2.9.4 L_1 最优化是中位数 (对比 L_2 平方和最优化是平均数)

命题：对实数集 $\{a_1, \dots, a_n\}$ ，函数 $f(x) = \sum_{i=1}^n |a_i - x|$ 在 x 取 $\{a_i\}$ 的中位数时达到最小值。

直觉证明：把 x 从极小往极大扫， f 的导数 (除若干断点外存在) 是 $\#\{a_i < x\} - \#\{a_i > x\}$ 。导数 $\leq 0 \Leftrightarrow \#\{a_i > x\} \geq \#\{a_i < x\}$ 即 x 在中位数下方；导数 ≥ 0 同理。两侧夹逼出“中位数处导数变号”。 n 偶时整段 $[a_{n/2}, a_{n/2+1}]$ 都是最小值点。

怎么用：题面看到“选一个 x 让 $\sum |a_i - x|$ 最小” / “把所有人移到同一个位置，移动总距离最小” / “涂色 / 校准 / 对齐成同一值的最小代价”都是中位数。 $O(N \log N)$ 排序取中位即可，不要傻乎乎地三分 / 二分。

对比 L_2 (平方和)： $\sum (a_i - x)^2$ 最小值点是平均值。中位数 vs 平均的根本差异就在 L_1 vs L_2 。

实战范例：NBUT 1569 HELLO - WORLD (最小化 $\sum |a_i - x|$ 模板题)、POJ 3579 Median (求所有数对绝对差的中位数，二分答案 + 双指针)、Gym 104386B Random Array (同模型变体)。

2.9.5 流式 / 增量中位数：对顶堆

要支持在线 insert + 实时查询当前中位数的场景，标准做法是大根堆 + 小根堆对顶维护：

- 下半堆 L ：大根堆，存较小的 $\lceil n/2 \rceil$ 个数；堆顶就是中位数 (n 奇时) 或下中位数 (n 偶时)。
- 上半堆 R ：小根堆，存较大的 $\lfloor n/2 \rfloor$ 个数；堆顶是上中位数。
- 不变量： $|L| - |R| \in \{0, 1\}$ 且 $\max L \leq \min R$ 。

insert 流程：先把新元素和 $\max L$ 比较丢进对应堆；若两堆 size 失衡 (差 ≥ 2) 就把多的那堆堆顶搬到

另一堆。每次 $O(\log n)$ 。**查询中位数**：直接 $\max L$ (n 奇) 或 $\frac{\max L + \min R}{2}$ (n 偶)， $O(1)$ 。

进阶：滑动窗口中位数也是同样的对顶堆 + 延迟删除 (lazy delete)，或者用 multiset + 迭代器维护。复杂度 $O(N \log K)$ 。

实战范例：Baekjoon 1655 说出中间值 (流式中位数模板)、HackerRank Find the Running Median、FZU 1702 query。

2.9.6 区间第 k 大 (中位数是 $k = \lceil \text{len}/2 \rceil$ 的特例)

静态 区间中位数 / 第 k 大查询，多次询问、不带修改：

主席树 (可持久化权值线段树) 预处理 $O(N \log N)$ 、单次 $O(\log N)$ ，最常用。

整体二分 二分答案 mid ，把所有 $\leq \text{mid}$ 的数加进 BIT 算每个询问区间内 $\leq \text{mid}$ 的个数，按是否 $\geq k$ 分流。 $O((N + Q) \log^2 N)$ 但常数小。

莫队 + 值域分块 $O((N + Q)\sqrt{N})$ ，写起来短，对询问规模友好。

动态 (带修改的区间中位数) 则要带修主席树 / 树套树，复杂度 $O(\log^2 N)$ 。

实战范例：SPOJ MKTHNUM K -th Number (主席树模板)、Baekjoon 7469 第 K 个数 (同款)。

2.10 常见结论 (conclusion)

2.10.1 约数个数定理

设正整数 n 的质因数分解为：

$$n = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$$

则 n 的约数个数 $d(n)$ 为：

$$d(n) = (e_1 + 1)(e_2 + 1) \cdots (e_k + 1)$$

例题：ABC-383-D 9 Divisors

要让 n 恰好有九个约数，由于 $9 = 9 \times 1 = 3 \times 3$ ，只有两种形式：

1. $n = p^8$ ：只有一个质因子，指数加一等于 9。
2. $n = p^2 \cdot q^2$ ($p \neq q$)：两个质因子， $(2+1)(2+1) = 9$ 。

2.10.2 Legendre 公式 ($n!$ 的质因子指数)

对质数 p ， $n!$ 中 p 的指数为

$$v_p(n!) = \sum_{i=1}^{\infty} \left\lfloor \frac{n}{p^i} \right\rfloor$$

求和到 $p^i > n$ 自动截断，实际只跑 $O(\log_p n)$ 项。

常见特例：

- **阶乘末尾零的个数**： $n!$ 末尾 0 的个数 = $\min(v_2(n!), v_5(n!)) = v_5(n!)$ (因为 $v_2 \geq v_5$)，即 $\lfloor n/5 \rfloor + \lfloor n/25 \rfloor + \lfloor n/125 \rfloor + \cdots$ 。

- **组合数中某质因子的指数**: $v_p\left(\binom{n}{k}\right) = v_p(n!) - v_p(k!) - v_p((n-k)!)$ 。结合 **Kummer 定理**: $v_p\left(\binom{n}{k}\right)$ 等于 p 进制下 $k + (n - k)$ 的进位次数。可用于快速判断 $\binom{n}{k}$ 的奇偶性 ($p = 2$ 时进位次数 $= 0 \iff \binom{n}{k}$ 是奇数, 进一步等价于 $k \& (n - k) = 0$, 即 k 的二进制是 n 的子集)。

闭式 (Legendre 等价形式): $v_p(n!) = \frac{n - s_p(n)}{p - 1}$, 其中 $s_p(n)$ 是 n 在 p 进制下各位数字之和。

2.10.3 二次方程根与二阶线性递推

若 α, β 都是某个二次方程 $x^2 = px + q$ 的根, 则 $S_n = k_1\alpha^n + k_2\beta^n$ 满足二阶线性递推:

$$S_n = p \cdot S_{n-1} + q \cdot S_{n-2}$$

经典例子: Fibonacci 数列 $F_n = \varphi^n + \psi^n$ ($\varphi, \psi = \frac{1 \pm \sqrt{5}}{2}$), 对应 $x^2 = x + 1$, 递推为 $F_n = F_{n-1} + F_{n-2}$ 。

例: $x_1, x_2 = 11 \pm 2\sqrt{30}$ 满足 $x^2 = 22x - 1$, 因此 $S_n = x_1^n + x_2^n$ 的递推为

$$S_n = 22S_{n-1} - S_{n-2}$$

2.10.4 逆序对奇偶性与置换环

交换一定改变逆序对数量的奇偶性。长度为 len 的循环移动一格等价于 $\text{len} - 1$ 次交换。

例题: [第 49 届 ICPC 区域赛沈阳站 D. Dot Product Game](#)

```
FOR(i, 1, N - 1) {
    char ch;
    ll l, r, d;
    cin >> ch >> l >> r >> d;
    ll len = r - l + 1;
    // 向左循环移动 d 次 = 交换 d*(len-1) 次
    if (d != 0)
        inva += (len - 1) * d;
    out();
}
```

快速判断排列逆序对的奇偶性: 将排列分解为置换环。一个长度为 k 的环需要 $k - 1$ 次交换复位, 因此逆序对数与 n 减去环的数量奇偶性相同。设排列有 c 个环, 则逆序对数的奇偶性等于 $n - c$ 的奇偶性。比直接用树状数组求逆序对好写。

3 数论

3.1 线性筛

完全理解线性筛

线性筛的作用就是为了保证每个合数都只被其【最小的】那个质因数标记一次。

对于一个将被标记的合数，我们记为 h 。在遍历过程中，它由当前的数字 i 和枚举到的素数 p 相乘得到：

$$h \leftarrow i \times p$$

线性筛的核心要求是，我们希望 p 恰好是 h 的最小素因数。

假设将 i 分解素因数，其分解形式为 $i = p_{i1} \times p_{i2} \times \dots$ ，其中 p_{i1} 是 i 的最小素因数，也就是代码中记录的 $\text{minp}[i]$ 。那么合数 h 的因数分解可以表示为：

$$h \leftarrow \underbrace{(p_{i1} \times p_{i2} \times \dots)}_i \times p$$

为了使 p 确实是 h 的最小素因数，必然要求 $p \leq p_{i1}$ ，即要求：

$$p \leq \text{minp}[i]$$

因此，在线性筛的内层循环中 (for p in primes)，我们首先执行：

```
minp[i * p] = p;
```

当遇到满足 $p == \text{minp}[i]$ 的情况时，说明当前的素数 p 已经达到了 i 的最小素因数边界。由于 primes 列表是递增的，如果继续往后枚举素数，后面的素数必定严格大于 $\text{minp}[i]$ ，这样新枚举的素数就不再是所生成合数 ($i \times p'$) 的最小素因数了。因此我们必须在此处执行 break，从而保证每个合数仅被其最小素因数筛除。

qoj 995868
cp-algorithms 线性筛

```
// minp[i] 存储的是 i 的最小素因数
// https://www.luogu.com.cn/record/221554377
vector<ll> minp, primes;
void sieve(ll n){
    minp.resize(n+5);
    // minp[1]=1; // 加了这句话可能把 1
    // 误判成素数，要小心
    for(int i=2; i<=n; ++i){
        if(minp[i]==0){ // 是素数
            minp[i]=i;
            primes.push_back(i);
        }
        for(auto p:primes){
            if(i*p>n){ // 超范围了
                break;
            }
            minp[i*p]=p;
            // 为了保证每个合数都只被其【最小的】
            // 那个质因数标记一次 break
            if(p==minp[i]){
                break;
            }
        }
    }
}
```

```
}
}
}
}

// 沿 minp 拆出 n 的素因子列表 (计重);
// 长度=Omega, sort+unique 后=omega
vector<ll> pFacLs(ll n) {
    if (n <= 0) return {};
    vector<ll> pLs;
    while (n != 1) {
        pLs.push_back(minp[n]);
        n /= minp[n];
    }
    return pLs;
}
```

使用线性筛的 minp 分解素因数更快，效率更高。用法 (如 CF2238D，答案 $\Omega + \omega - 1$):

```
vector<ll> pLs = pFacLs(N); //
// 素因子列表 (计重)
ll pCt = pLs.size(); // Omega =
// 列表长度
sort(all(pLs));
ll sz = unique(all(pLs)) - pLs.begin(); //
// omega = 去重后长度
```

3.2 Miller-Rabin 与 Pollard's Rho

模板题：洛谷 P4718【模板】Pollard-Rho (AC 提交)。

$\text{is_prime}(n)$ 用前 7 个固定底数做确定性 Miller-Rabin，覆盖整个 unsigned long long 范围。 $\text{factor}(n)$ 用 Pollard's Rho 把 n 拆成质因子，期望复杂度 $O(n^{1/4})$ 。

速度关键点：(a) 浮点估商 mulmod 绕开 __int128 的硬件 DIV (x86-64 上快 $\sim 2x$ ，要求 $n < 2^{63}$)；(b) Pollard 进入前先小质数试除 ≤ 23 ，把“含小因子的合数”秒掉；(c) $f(x) = x^2 + 1$ Floyd 双指针，差值批乘 256 步再 gcd 一次 (gcd 比 mulmod 慢一个量级，必须摊)；(d) 撞环用计数器 tot++ 重启，不用随机库；(e) 累乘归零时保留旧 p 。

```
// 直接用模板头的 ll / ull, 不另外 using
ull mulmod(ull a, ull b, ull m) { // 要求 m <
    // 2^63
    ull q = (long double)a * b / m;
    ll r = (ll)(a * b - m * q);
    return r < 0 ? r + m : ((ull)r >= m ? r - m :
        // r);
}

ull powmod(ull a, ull b, ull m) {
    ull r = 1 % m; a %= m;
    for (; b >= 1, a = mulmod(a, a, m)) if (b
        // & 1) r = mulmod(r, a, m);
    return r;
}

bool is_prime(ull n) {
    if (n < 2) return false;
```



```

for (ull p :
    ↪ {2,3,5,7,11,13,17,19,23,29,31,37})
    if (n % p == 0) return n == p;
int s = __builtin_ctzll(n - 1);
ull d = (n - 1) >> s;
for (ull a : {2ULL,325ULL,9375ULL,28178ULL,45
    ↪ 0775ULL,9780504ULL,1795265022ULL}) {
    ull x = powmod(a % n, d, n);
    if (x <= 1 || x == n - 1) continue;
    bool composite = true;
    for (int i = 0; i < s - 1; ++i) {
        x = mulmod(x, x, n);
        if (x == n - 1) { composite = false;
            ↪ break; }
    }
    if (composite) return false;
}
return true;
}
ull pollard(ull n) {
    for (ull p : {2,3,5,7,11,13,17,19,23}) if (n
        ↪ % p == 0) return p;
    static ull tot = 0;
    auto f = [&](ull x) { x = mulmod(x, x, n) +
        ↪ 1; return x >= n ? x - n : x; };
    ull x = 0, y = 0, p = 1, q, g;
    for (int i = 0; (i & 255) || (g = __gcd(p,
        ↪ n)) == 1; ++i, x = f(x), y = f(f(y))) {
        if (x == y) { x = tot++; y = f(x); } //
            ↪ 撞环, 换起点重启
        q = mulmod(p, x > y ? x - y : y - x, n);
        if (q) p = q; //
            ↪ 累乘归零则保留旧 p
    }
    return g;
}
void factor(ull n, vector<ull>& res) { // n < 2
    ↪ 时什么都不写入; res 计重、无序、不清空
    if (n < 2) return;
    if (is_prime(n)) { res.push_back(n); return; }
    ull d = pollard(n);
    factor(d, res); factor(n / d, res);
}

```

用法 (P4718 原题: n 是素数输出 Prime, 否则输出最大质因子):

```

if (is_prime(N)) puts("Prime");
else {
    vector<ull> res; // 出参: 计重、无序,
    ↪ 多测每组自己 clear
    factor(N, res);
    printf("%llu\n", *max_element(all(res)));
}

```

拿到 `res` 后: 长度即 $\Omega(n)$ (计重), 排序去重后的长度即 $\omega(n)$; 用 `map<ull,int>` 数出各质因子指数 c_p 后, 约数个数 $d(n) = \prod (c_p + 1)$, 欧拉函数 $\varphi(n) = \prod p^{c_p-1}(p-1)$ 。

注意: $n \leq 2^{63}$ 是硬约束 (浮点估商会崩), 要覆盖 $[2^{63}, 2^{64}]$ 就把 `mulmod` 换成 `return ((__int128)a * b % m;`, 慢约 2 倍但无范

围限制; $n \leq 10^6$ 或要对整段区间分解时, 直接用上面线性筛的 `minp` 更快。

3.3 欧拉定理, 扩展欧拉定理, 欧拉函数的求解 (线性筛, 试除法)

完全理解欧拉函数的计算和欧拉定理 (使用线性筛求解) (使用试除法求解)

定义: 欧拉函数 $\varphi(n)$ 表示小于或等于 n 且与 n 互质的正整数的个数。

性质与定理:

1. **质数的欧拉函数:** 若 p 是质数, 则 $\varphi(p) = p - 1$ 。

解释: 因为 p 是质数, 所以在 $1 \sim p$ 中, 除了 p 本身以外, 其余的 $p - 1$ 个数都与 p 互质。

2. **质数次幂的欧拉函数:** 若 p 是质数, 则 $\varphi(p^k) = p^k - p^{k-1} = p^{k-1}(p - 1)$ 。

解释: 在 $1 \sim p^k$ 中, 与 p^k 不互质的数只能是 p 的倍数。由于 $p^k = p^{k-1} \times p$, 所以在该范围内 p 的倍数共有 p^{k-1} 个 (包含 p^k 本身)。将其减去, 剩下的就是与 p^k 互质的数。

3. **积性函数性质:** 若 $\gcd(m, n) = 1$, 则 $\varphi(mn) = \varphi(m)\varphi(n)$ 。

解释: 由中国剩余定理 (CRT) 可知, 任意一个与 mn 互质的数, 都可以唯一映射到一个与 m 互质的数和与 n 互质的数构成的二元组。因此满足条件的数的个数就是这两者的乘积。

4. **欧拉函数通式:** 若 n 的质因数分解为 $n = p_1^{k_1} p_2^{k_2} \dots p_m^{k_m}$, 则 $\varphi(n) = n \times (1 - \frac{1}{p_1}) \times (1 - \frac{1}{p_2}) \dots \times (1 - \frac{1}{p_m})$ 。

解释: 由积性函数性质和质数次幂的公式直接推导得出。这是我们在使用试除法 ($O(\sqrt{n})$ 求单个数的欧拉函数) 时的理论基础。

5. **欧拉定理:** 若 $\gcd(a, m) = 1$, 则 $a^{\varphi(m)} \equiv 1 \pmod{m}$ 。

解释: 当 m 为质数时, $\varphi(m) = m - 1$, 该定理即退化为常见的费马小定理 $a^{m-1} \equiv 1 \pmod{m}$ 。

6. **扩展欧拉定理 (降幂公式):**

$$a^b \equiv \begin{cases} a^{b \bmod \varphi(m)} \pmod{m}, & \gcd(a, m) = 1 \\ a^b \pmod{m}, & \gcd(a, m) \neq 1, b < \varphi(m) \\ a^{b \bmod \varphi(m) + \varphi(m)} \pmod{m}, & \gcd(a, m) \neq 1, b \geq \varphi(m) \end{cases}$$

解释: 常用于解决指数极大的取模降幂问题 (如计算高塔求幂 $a^{b^c} \pmod{m}$), 此公式在 a, m 不互质的情况下依然成立, 非常实用。

3.3.1 欧拉定理与费马小定理的证明

对于任意正整数 m 和整数 a , 若 $\gcd(a, m) = 1$, 则

$$a^{\varphi(m)} \equiv 1 \pmod{m}$$

其中 $\varphi(m)$ 是欧拉函数, 表示 1 到 m 中与 m 互素的正整数个数。

证明: 考虑模 m 的缩系 S (1 到 m 中所有与 m 互素的数, 共 $\varphi(m)$ 个)。当 $\gcd(a, m) = 1$ 时, S 中每个元素乘以 a 后模 m 仍是 S 的一个排列 (双射), 于是

$$\prod_{x \in S} (ax) \equiv \prod_{x \in S} x \pmod{m}$$

左边提出 a :

$$a^{\varphi(m)} \cdot \prod_{x \in S} x \equiv \prod_{x \in S} x \pmod{m}$$

因为 $\prod_{x \in S} x$ 与 m 互素, 存在逆元, 两边消掉即得 $a^{\varphi(m)} \equiv 1 \pmod{m}$ 。

费马小定理: 当 $m = p$ 为素数时, $\varphi(p) = p - 1$, 即 $a^{p-1} \equiv 1 \pmod{p}$ 。

线性筛求欧拉函数的原理:

利用欧拉函数的几条重要性质, 我们可以在 $O(n)$ 的线性时间内筛出 $1 \sim n$ 所有数的欧拉函数, 递推原理如下:

1. 当 i 为质数时: 根据性质 1 可知 $\varphi(i) = i - 1$ 。
2. 当 $i \bmod p \neq 0$ 时: 由于 p 是质数, 说明 i 和 p 互质。根据积性函数性质, 有 $\varphi(i \times p) = \varphi(i) \times \varphi(p) = \varphi(i) \times (p - 1)$ 。
3. 当 $i \bmod p = 0$ 时: 说明 p 已经是 i 的一个质因子, 因此 $i \times p$ 的质因子种类与 i 完全相同。根据欧拉函数通式:

$$\varphi(i) = i \prod \left(1 - \frac{1}{p_j}\right)$$

因为质因子种类没变, 所以通式后面的连乘部分 $\prod \left(1 - \frac{1}{p_j}\right)$ 完全一样, 只是把前面的系数 i 变成了 $i \times p$:

$$\varphi(i \times p) = i \times p \prod \left(1 - \frac{1}{p_j}\right) = \varphi(i) \times p$$

```
#include <bits/stdc++.h>
using namespace std;

struct LinearSievePhi {
    int n;
    vector<int> primes, minp, phi;

    LinearSievePhi(int n = 0) {
        if (n > 0) init(n);
    }

    void init(int m) {
        n = m;
```

```
primes.clear();
minp.assign(n + 1, 0);
phi.assign(n + 1, 0);

phi[1] = 1;
for (int i = 2; i <= n; i++) {
    if (minp[i] == 0) { // i 是一个质数
        minp[i] = i;
        primes.push_back(i);
        phi[i] = i - 1; // 质数的欧拉函数性质
    }
    for (int p : primes) {
        if (1LL * i * p > n) break;
        minp[i * p] = p;
        if (i % p == 0) {
            // p 已经是 i 的质因子, i*p 和 i
            // 的质因子集合完全相同
            phi[i * p] = phi[i] * p;
            break; // 保证每个数只被其最小质因子筛去
        } else {
            // i 和 p 互质, 利用积性函数性质:
            // phi(i*p) = phi(i) * phi(p)
            phi[i * p] = phi[i] * (p - 1);
        }
    }
}
```

```
/*
 * 试除法求一个数的欧拉函数 (求 [1, n] 内与 n
 * 互质的数的个数)
 * 2026-03-13: AC https://codeforces.com/gym/10
 * 6380/submission/366492238
 */
long long euler_phi(long long n) {
    long long res = n;
    // 枚举可能存在的质因子, 直到 sqrt(n)
    for (long long p = 2; p * p <= n; p++) {
        // 由于合数因子必定被其更小的质因子提前除尽,
        // 因此能进入 if 的 p 必然是质数
        if (n % p == 0) {
            // 根据通式: res = res * (1 - 1/p) = res /
            // p * (p - 1)
            res = res / p * (p - 1);
            // 剔除 n 中所有质因子 p
            while (n % p == 0) n /= p;
        }
    }
    // 若循环结束 n 仍大于 1, 说明存在唯一一个大于
    // sqrt(n) 的质因子
    if (n > 1) {
        res = res / n * (n - 1);
    }
    return res;
}
```

3.4 gcd (迭代与递归)

不过其实我们在 g++ 下有 `__gcd` 函数, 可以直接使用。如果是在 C++17 以及之后, 我们也有这个 `std::gcd` 函数, 以及 `std::lcm` 函数。

```
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
```

```
inline ll gcd(ll a, ll b) {
    if (a < b) swap(a, b);
    while (b) {
        a %= b;
        swap(a, b);
    }
    return a;
}
```

3.5 exgcd 与逆元

求解 $ax + by = \gcd(a, b)$ 。利用 exgcd 可以求 a 在模 p 意义下的逆元 (要求 $\gcd(a, p) = 1$)。

验证题目: P2613 【模板】有理数取余

```
// AC https://www.luogu.com.cn/record/273130120
void exgcd(ll a, ll b, ll &x, ll &y) {
    if (!b) { x = 1; y = 0; return; }
    exgcd(b, a % b, y, x);
    y -= a / b * x;
}

ll inv_exgcd(ll a, ll mod) {
    ll x = 0, y = 0;
    exgcd(a % mod, mod, x, y);
    x %= mod;
    if (x < 0) x += mod;
    return x;
}
```

3.6 在线快速逆元 ($O(1)$ 查询)

QOJ XXII 赛前模板训练赛 Problem 5

基于 Stern-Brocot 树的 $O(1)$ 在线模逆元查询。预处理 $O(2^{21})$ 时间与空间 (约 20 MB)，每次查询 $O(1)$ 。

原理: 设模数为 p 。对于输入 $x \in [1, p-1]$ ，利用 Stern-Brocot 树预处理的分数近似表，找到分数 a/b (分母 $b < 2048$)，使得 $a/b \approx x/p$ 。令 $r = xb - a \cdot p$ ，因为 $a \cdot p \equiv 0$ ，有 $xb \equiv r \pmod{p}$ ，从而：

$$x^{-1} \equiv b \cdot r^{-1} \pmod{p}$$

$|r| \leq 2^{21}$ ，故 r^{-1} 可从预计算的小值逆元表中 $O(1)$ 查找。

约束: 模数 p 必须为质数且 $2^{21} < p < 2^{30}$ (覆盖 998244353 和 $10^9 + 7$)。

```
// AC https://qoj.ac/submission/2213840
constexpr bool is_prime_constexpr(int n) {
    if (n == 998244353 || n == 1000000007) return true;
    for (int i = 2; (ll)i * i <= n; i++)
        if (n % i == 0) return false;
    return true;
}
```

```
}

template<int mod>
struct FastInv {
    static_assert(2 * (1 << 20) < mod && mod < (1
        << 30));
    static_assert(is_prime_constexpr(mod));

    // Stern-Brocot 树构建分数近似表
    static vector<pair<uint16_t, uint16_t>>
        build_frac() {
        vector<pair<uint16_t, uint16_t>> res(1 <<
            << 20);
        array<tuple<int, int, int, int>, 2048> st;
        int p = 0;
        st[p++] = {0, 1, 1, 1};
        while (p) {
            auto [a, b, c, d] = st[--p];
            if (b + d < 2048) {
                st[p++] = {a + c, b + d, c, d};
                st[p++] = {a, b, a + c, b + d};
            } else {
                int s = (ll)a * mod / (1024 * b);
                int t = (ll)c * mod / (1024 * d);
                res[s] = {a, b};
                res[t] = {c, d};
                if (a > c) a = c;
                if (b > d) b = d;
                for (int i = s + 1; i < t; i++)
                    res[i] = {a, b};
            }
        }
        return res;
    }

    // 线性递推求小值逆元 (同时覆盖正负值)
    static vector<uint32_t> build_sinv() {
        constexpr int M = 2 * (1 << 20);
        vector<uint32_t> res(4 * (1 << 20) + 1);
        res[M + 1] = 1;
        res[M - 1] = mod - 1;
        for (int i = 2; i <= M; i++) {
            uint32_t x = (ll)(mod - res[M + (mod %
                << i)])
                * (mod / i) % mod;
            res[M + i] = x;
            res[M - i] = mod - x;
        }
        return res;
    }

    inline static vector<pair<uint16_t,
        << uint16_t>> frac
        = build_frac();
    inline static vector<uint32_t> sinv =
        build_sinv();

    static int inv(unsigned x) {
        auto [a, b] = frac[x >> 10];
        return (ll)sinv[2 * (1 << 20) + x * b
            - (unsigned)a * mod] * b % mod;
    }
};
```

```
// 使用: FastInv<998244353>::inv(x)
// 或   FastInv<(int)1e9+7>::inv(x)
```

调用示例: 静态成员在 main 之前自动完成预处理, 无需手动初始化。

```
// 直接调用, 无需 init
ll ans = FastInv<mod>::inv(x);    // x 的逆元

// 也可以包一层方便使用
inline ll inv(ll x) {
    return FastInv<mod>::inv(x);
}

// 除法: a / b (mod p) = a * inv(b) % p
ll res = a % mod * inv(b) % mod;
```

适用场景: 当题目中需要大量求逆元, 且快速幂的 $O(\log p)$ 常数成为瓶颈时, 可以用本模板替换。典型场景包括:

- **在线逆元查询:** 交互题中每次给定 x 要求实时返回 x^{-1} , 不允许离线预处理所有值。
- **NTT 内部加速:** NTT 蝴蝶变换中需要大量求逆元 (如求原根的逆元、长度的逆元等), 用 $O(1)$ 逆元可降低常数。
- **组合数 / 多项式运算的内层循环:** 当逆元调用次数达到 $10^7 \sim 10^8$ 量级时, 快速幂的 30 次乘法会成为瓶颈, 而本模板仅需 1 次查表 + 1 次乘法。
- **无法预处理阶乘逆元的情形:** 如果逆元的输入没有规律 (不是连续的 $1 \sim n$), 无法用阶乘逆元表一次性预处理, 此时本模板是最优选择。

与其他求逆元方法的对比:

- 快速幂 $O(\log p)$: 无需预处理, 但单次查询慢, 适合逆元调用次数少的场景。
- 线性递推逆元 $O(n)$ 预处理 + $O(1)$ 查询: 只能处理 $1 \sim n$ 的连续逆元, 且 n 不能太大。
- 阶乘逆元表: 适合组合数场景, 但只能求阶乘的逆元, 不能直接求任意数的逆元。
- **本模板:** $O(2^{21})$ 预处理后任意 $x \in [1, p-1]$ 均可 $O(1)$ 查询, 代价是约 20 MB 内存。

3.7 中国剩余定理 (CRT)

给定 k 个两两互素的正整数 $m_1, m_2, \dots, m_k$, 以及同余方程组:

$$\begin{cases} x \equiv r_1 \pmod{m_1} \\ x \equiv r_2 \pmod{m_2} \\ \vdots \\ x \equiv r_k \pmod{m_k} \end{cases}$$

则在模 $M = m_1 \times m_2 \times \dots \times m_k$ 意义下, x 有唯一解:

$$x \equiv \sum_{i=1}^k r_i \cdot M_i \cdot M_i^{-1} \pmod{M}$$

其中 $M_i = M/m_i$, M_i^{-1} 是 M_i 在模 m_i 意义下的逆元 (即 $M_i \cdot M_i^{-1} \equiv 1 \pmod{m_i}$)。

直觉: $M_i \cdot M_i^{-1}$ 就是那个“只在模 m_i 下为 1、在其他模下为 0”的基础解, 公式 $\sum r_i \cdot M_i \cdot M_i^{-1}$ 是把每个余数 r_i 按对应基础解线性组合。

```
// 依赖 exgcd 求逆元
void exgcd(ll a, ll b, ll &x, ll &y) {
    if (!b) { x = 1; y = 0; return; }
    exgcd(b, a % b, y, x);
    y -= a / b * x;
}

ll inv_mod(ll a, ll m) {
    ll x = 0, y = 0;
    exgcd(a % m, m, x, y);
    return (x % m + m) % m;
}

// r[i]: 余数, m[i]: 模数 (两两互素)
ll crt(vector<ll> &r, vector<ll> &m) {
    int k = r.size();
    ll M = 1;
    for (int i = 0; i < k; ++i) M *= m[i];
    ll x = 0;
    for (int i = 0; i < k; ++i) {
        ll Mi = M / m[i];
        ll ti = inv_mod(Mi, m[i]);
        x = (x + r[i] % M * (Mi % M) % M * (ti % M)) % M;
    }
    return (x + M) % M;
}
```

3.7.1 CRT 双射形式 (一维循环 $\leftrightarrow$ 二维网格)

$\gcd(n, m) = 1$ 时, 映射 $i \mapsto (i \bmod n, i \bmod m)$ 是 $\mathbb{Z}/(nm) \rightarrow \mathbb{Z}/n \times \mathbb{Z}/m$ 的双射, 即让 i 跑遍 $[0, nm)$, 二元组 $(i \bmod n, i \bmod m)$ 恰好覆盖网格 $[0, n) \times [0, m)$ 每个格点一次。这就是 CRT 的同构 $\mathbb{Z}/(nm) \cong \mathbb{Z}/n \times \mathbb{Z}/m$ 反过来读——常用于把一维 $O(nm)$ 循环拆成两个独立的 $O(n) + O(m)$ 循环 (如欧拉函数积性 $\varphi(nm) = \varphi(n)\varphi(m)$ 的证明)。详细证明、几何图示与 $\gcd > 1$ 时像集的刻画见 TemplateDetailedExplain/CRT-双射形式.tex。

例题: QOJ 9799 *Magical Palette* (乘积分量化)、QOJ 2735 *Shifty Grid* (行列循环位移)。

3.7.2 扩展中国剩余定理 (exCRT)

当模数 $m_1, m_2, \dots, m_k$ 不一定两两互素时, 普通 CRT 不再适用。exCRT 通过逐步合并方程来求解: 每次将两个同余方程合并为一个, 直到只剩一个方程。

设当前已合并的结果为 $x \equiv r_1 \pmod{M_1}$, 下一个方程为 $x \equiv r_2 \pmod{M_2}$ 。令 $x = M_1 \cdot t + r_1$, 代入得:

$$M_1 \cdot t \equiv r_2 - r_1 \pmod{M_2}$$

设 $g = \gcd(M_1, M_2)$, $c = r_2 - r_1$ 。若 $g \nmid c$ 则无解；否则两边除以 g , 用 `exgcd` 求出 t , 合并后新的模数为 $\text{lcm}(M_1, M_2)$ 。

```
// 返回 {r, M}, 表示同余  $x \equiv r \pmod{M}$ 
// 无解返回 {-1, -1}
pair<ll, ll> exCRT(vector<ll> &r, vector<ll> &m)
{
    ll cur = r[0], M = m[0];
    for (int i = 1; i < (int)r.size(); ++i) {
        ll r2 = r[i], m2 = m[i];
        ll g = __gcd(M, m2);
        ll c = (r2 - cur % m2 + m2) % m2;
        if (c % g) return {-1, -1};
        // 解同余  $M/g * t \equiv c/g \pmod{m2/g}$ 
        ll x0 = 0, y0 = 0;
        exgcd(M / g, m2 / g, x0, y0);
        ll mod2 = m2 / g;
        ll t = (__int128)x0 * (c / g) % mod2;
        t = (t + mod2) % mod2;
        cur = cur + (__int128)M * t % (M / g * m2);
        M = M / g * m2; // lcm(M, m2)
        cur = ((cur % M) + M) % M;
    }
    return {cur, M};
}
```

多元情形可对 n 归纳: $\gcd(a_1, \dots, a_n) = \gcd(\gcd(a_1, \dots, a_{n-1}), a_n)$, 逐对调用二元结论即可。

3.8.1 裴蜀定理的典型应用

1. 判定一次丢番图方程是否有解: 方程 $ax + by = c$ 有整数解 $(x, y) \in \mathbb{Z}^2$ 当且仅当 $\gcd(a, b) \mid c$ 。 n 元推广: 方程

$$a_1x_1 + a_2x_2 + \dots + a_nx_n = c$$

有整数解 $(x_1, \dots, x_n) \in \mathbb{Z}^n$ 当且仅当 $\gcd(a_1, a_2, \dots, a_n) \mid c$ 。这是最直接的判定层应用, 常用作其它构造题的可行性预判。例题: [HDU 2026 春季联赛 7 1001 - Card](#) (图上选点喝药水拼魔力值 V , 可达性归约为“路径上点的 $\gcd$ 整除 V ”, 配合 (节点, $\gcd$) 分层图 Dijkstra 求最短路)。

3.7.3 CRT 的典型应用

1. 对多个质数分别取模后还原: 当答案可能很大 (如高精度), 但已知上界 B 时, 选取若干质数 $p_1, p_2, \dots$ 使得 $\prod p_i > B$, 对每个 p_i 分别计算答案 r_i , 最后用 CRT 还原出真实答案。

2. Lucas 定理 + CRT: 当模数 m 不是质数但可以分解为 $m = p_1^{e_1} \times p_2^{e_2} \times \dots$ 时, 对每个 $p_i^{e_i}$ 分别求 $C_n^k \bmod p_i^{e_i}$, 再用 CRT 合并。

3. 同余方程组求解: 直接的应用场景, 如“一个数除以 3 余 2, 除以 5 余 3, 除以 7 余 2, 求这个数”。

3.8 裴蜀定理 (Bézout's identity)

裴蜀定理 (Bézout's identity): 对整数 a, b (不全为零), 方程 $ax + by = c$ 存在整数解 (x, y) 当且仅当 $\gcd(a, b) \mid c$ 。

多元推广: 对整数 $a_1, a_2, \dots, a_n$ (不全为零), 方程 $\sum_{i=1}^n a_i x_i = c$ 存在整数解 $(x_1, \dots, x_n)$ 当且仅当 $\gcd(a_1, a_2, \dots, a_n) \mid c$ 。

等价表述: $\{a_1, \dots, a_n\}$ 的全体整数线性组合恰好等于 $\gcd(a_1, \dots, a_n)$ 的全体整数倍:

$$\left\{ \sum_{i=1}^n k_i a_i : k_i \in \mathbb{Z} \right\} = \gcd(a_1, \dots, a_n) \cdot \mathbb{Z}.$$

证明梗概 (二元):

- ($\Rightarrow$) 设 $g = \gcd(a, b)$ 。由 $g \mid a$ 、 $g \mid b$ 得 $g \mid (ax + by) = c$ 。
- ($\Leftarrow$) 由欧几里得算法构造性给出 (即 `exgcd`)。先求 $ax_0 + by_0 = g$ 的一组解, 两边乘 c/g 即得 $ax + by = c$ 的一组解。

3.9 莫比乌斯反演 (Möbius inversion)

3.9.1 莫比乌斯函数 μ

莫比乌斯函数 $\mu(n)$ 定义为:

$$\mu(n) = \begin{cases} 1, & n = 1, \\ (-1)^k, & n = p_1 p_2 \cdots p_k \text{ (} k \text{ 个互异质因子, 无平方因子)}, \\ 0, & n \text{ 含平方因子 (存在质数 } p \text{ 使 } p^2 \mid n) \text{ .} \end{cases}$$

μ 是积性函数 ($\gcd(a, b) = 1 \Rightarrow \mu(ab) = \mu(a)\mu(b)$), 可在线性筛里顺带在 $O(n)$ 内预处理: 遇到质数 p 令 $\mu(p) = -1$; 筛到 $i \cdot p$ 时, 若 $p \mid i$ 则 $i \cdot p$ 含平方因子 p^2 , $\mu(i \cdot p) = 0$, 否则 $\mu(i \cdot p) = -\mu(i)$ 。

```
const int N = 1e6 + 5;
int mu[N], primes[N], cnt;
bool notPrime[N];
void sieve(int n) {
    mu[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!notPrime[i]) { primes[cnt++] = i; mu[i] = -1; }
        for (int j = 0; j < cnt && 1LL * i * primes[j] <= n; j++) {
            notPrime[i * primes[j]] = true;
            if (i % primes[j] == 0) { mu[i * primes[j]] = 0; break; }
            mu[i * primes[j]] = -mu[i];
        }
    }
}
```

3.9.2 反演公式

因数 (约数) 形式: 若对任意正整数 n 都有

$$F(n) = \sum_{d|n} f(d),$$

则

$$f(n) = \sum_{d|n} \mu\left(\frac{n}{d}\right) F(d) = \sum_{d|n} \mu(d) F\left(\frac{n}{d}\right).$$

倍数形式: 若在有限 (或收敛) 范围内对任意正整数 d 都有 $F(d) = \sum_{d|n} f(n)$ (n 取遍 d 的所有倍数), 则

$$f(d) = \sum_{d|n} \mu\left(\frac{n}{d}\right) F(n).$$

下面给出证明 (主体推导为手写稿)。核心是关键引理与一次求和次序交换; 后者另配一张 $n = 12$ 的示意图辅助理解。

3.9.3 关键引理: $\sum_{d|n} \mu(d) = [n = 1]$

$$\sum_{d|n} \mu(d) = [n = 1] = \begin{cases} 1, & n = 1, \\ 0, & n > 1. \end{cases}$$

$n = 1$ 显然成立。 $n > 1$ 时设其有 k 个互异质因子, 只有无平方因子的约数 (从 k 个质因子中不重复选取) 贡献非零 μ , 按选取个数分组并用二项式定理即得 $(1 - 1)^k = 0$:

证明 $\sum_{d|n} \mu(d) = 0$

$p_1, p_2, \dots, p_k$ 个质因数

d 中重复选取 $\mu(d) = 0$, 因此需不重复选取

$$C_k^0 - C_k^1 + C_k^2 - C_k^3 + C_k^4 + \dots + (-1)^k C_k^k$$

$$= (1-1)^k = 0$$

使用了二项式定理

3.9.4 反演公式（因数形式）的证明

命题. 已知 $f(n) = \sum_{d|n} g(d)$, 要证 $g(n) = \sum_{d|n} \mu(n/d) f(d)$:

已知: $f(n) = \sum_{d|n} g(d)$

要证: $g(n) = \sum_{d|n} \mu\left(\frac{n}{d}\right) f(d)$

代入并交换求和次序. 把右边的 $f(d)$ 代入为 $\sum_{k|d} g(k)$, 交换求和次序后令 $x = n/d$, 内层中括号化为 $\sum_{x|n/k} \mu(x)$:

$$\begin{aligned}
 \text{右边} &= \sum_{d|n} \mu\left(\frac{n}{d}\right) \left[\sum_{k|d} g(k) \right] \\
 &= \sum_{k|n} g(k) \left[\sum_{d|n \text{ 且 } k|d} \mu\left(\frac{n}{d}\right) \right] \\
 \text{令 } x &= \frac{n}{d}, \quad x = \frac{n}{c \cdot k}, \text{ 即 } cx = \frac{n}{k}, x | \frac{n}{k} \\
 \text{中括号} &= \sum_{x | \frac{n}{k}} \mu(x)
 \end{aligned}$$

用 μ 的性质收束. 由关键引理 $\sum_{x|n/k} \mu(x) = [n/k = 1] = [k = n]$, 于是右边 $= \sum_{k|n} g(k) [k = n] = g(n) =$ 左边, 得证:

使用 μ 的性质

$$\text{右边} = \sum_{k|n} g(k) \times \begin{cases} 1 & \text{当 } k=n \text{ 即 } \frac{n}{k}=1 \\ 0 & \text{当 } k < n \text{ 即 } \frac{n}{k} \neq 1 \end{cases}$$

$$\text{右边} = g(n) = \text{左边}$$

得证

求和次序交换示意 ($n=12$). 上面「交换求和次序」一步, 可用约数 incidence 网格直观理解——原式与交换后覆盖的是同一批 (k, d) 点对:

$k =$	1	2	3	4	6	12
$d=1$	•					
$d=2$	•	•				
$d=3$	•		•			
$d=4$	•	•		•		
$d=6$	•	•	•		•	
$d=12$	•	•	•	•	•	•

求和次序交换 ($n = 12$, 约数 $\{1, 2, 3, 4, 6, 12\}$): 实心点表示 $k \mid d$ 。蓝行是原式「固定 $d = 6$ 、横向对 $k \mid d$ 求和」; 橙列是交换后「固定 $k = 3$ 、纵向对满足 $k \mid d \mid n$ 的 d 求和」。两种方式覆盖的是同一批点, 故可交换。

4 计数与组合

4.1 Catalan 数 (合法括号串数量)

Catalan 数是一类专门计数“前缀不越界”结构的经典数列。第 n 个 Catalan 数为:

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}.$$

其中组合数相减的写法可以用反射法理解: 先不管前缀限制, 只要求 n 个左括号和 n 个右括号, 总数是 $\binom{2n}{n}$ 。坏串是某个前缀中右括号第一次严格多于左括号的串。找到第一次使前缀和变为 -1 的位置, 把从开头到这个位置的括号全部翻转, 例如 $()()()$ 在第 3 位第一次出错, 翻转前三位得到 $)(((()$ 。这样坏串就会一一对应到含有 $n+1$ 个左括号、 $n-1$ 个右括号的长度 $2n$ 串。反过来, 对任意这样的串, 找到第一次前缀和变为 1 的位置并翻转前缀, 就能还原唯一的坏串。因此坏串数量是 $\binom{2n}{n+1}$, 合法串数量即为 $\binom{2n}{n} - \binom{2n}{n+1}$ 。

最经典的含义是: 长度为 $2n$ 的合法括号串数量。这里合法括号串要求恰好有 n 个左括号和 n 个右括号, 并且任意前缀中左括号数量都不少于右括号数量。换成前缀和语言, 就是把左括号看成 $+1$ 、右括号看成 -1 , 要求总和为 0 且任意前缀和都不为负。

例如 $n = 3$ 时, 合法括号串共有 $C_3 = 5$ 个:

$((()))$, $(()())$, $(())()$, $()(())$, $()()()$ 。

竞赛中遇到类似模型时, 不要只凭“像括号”就直接套 Catalan 数; 先检查三个条件: 对象能否编码成 $+1/-1$ 序列, 最终总和是否固定, 合法性是否只由前缀和下界决定。若终点为 0 、下界为 0 , 就是标准 Catalan 数; 若终点或下界发生偏移, 通常要改用 Ballot 数或反射法的推广。合法括号串、栈的入栈/出栈过程、不过对角线的网格路径, 都是同一个前缀约束的不同外壳。

4.2 求组合数 (递推法)

会推这个杨辉三角吗? 那你就会推组合数, 一样的东西。

```
// c[a][b] 表示从 a 个苹果中选 b 个的方案数
for (int i = 0; i < N; i++)
    for (int j = 0; j <= i; j++)
        if (!j) c[i][j] = 1;
        else c[i][j] = (c[i-1][j] + c[i-1][j-1]) % mod;
```

4.3 求组合数 (逆元)

增加了这个如果传入参数有误 (比如说顺序错了), 输出报错信息的功能。

Comb(x) 初始化不能够传入 **mod**, 最大传入 $\text{Comb}(\text{mod}-1)$, 传入 mod 会导致 $\text{frac}[\text{mod}] = 0$, 进而导致所有的 infrac 变成 0 。

关于阶乘逆元的递推求法: 由阶乘的定义可知:

$$\text{frac}_{i+1} = \text{frac}_i \times (i+1)$$

在模 mod 意义下, 等式两边同乘 $\text{infrac}_{i+1} \times \text{infrac}_i$ 进行化简, 直接可得递推式:

$$\text{infrac}_i \equiv \text{infrac}_{i+1} \times (i+1) \pmod{\text{mod}}$$

因此, 只需要用费马小定理先算出最大项 $n!$ 的逆元, 便可 $O(n)$ 倒推算出所有的阶乘逆元。

```
namespace Comb{ // 组合数加逆元,
    ↪ 依赖于这个MAXN (初始化)
    vector<ll> frac, infrac;
    int ln=1;
    bool initialized=false;
    // 在 k=0 或者更小的时候 return 1
    inline ll binpow(ll a, ll k) {
        ll res = 1;
        a %= mod;
        while (k > 0) {
            if (k & 1) res = (res * a) % mod;
            a = (a * a) % mod;
            k >>= 1;
        }
        return res;
    }

    void Comb(int n) {
        ln = n;
        frac.assign(n + 5, 1);
        infrac.assign(n + 5, 1);
        for (int i = 1; i <= ln; ++i) {
            frac[i] = (frac[i-1] * i) % mod;
        }
        infrac[ln] = binpow(frac[ln], mod - 2);
        for (int i = ln - 1; i >= 0; --i) {
            infrac[i] = (infrac[i+1] * (i+1)) %
            ↪ mod;
        }
    }

    inline ll CC(ll n, ll k) {
        if (k < 0 || k > n) {
            #ifdef LOCAL
                cerr<<"CC 组合数传入参数不合法,
                ↪ 可能是传入顺序错了\n";
            #endif
            return 0; // Return 0 if k is out of range
            ↪ [0, n]
        }
        if (!initialized){
            Comb(MAXN);
            initialized=true;
        }
        ll res = frac[n];
```

```

    res = (res * infrac[n - k]) % mod;
    res = (res * infrac[k]) % mod;
    return res;
}

inline ll PP(ll n, ll k) {
    if (k < 0 || k > n) {
#ifdef LOCAL
        cerr<<"PP 排列数传入参数不合法，
        ↪ 可能是传入顺序错了\n";
#endif
        return 0;
    }
    if(!initialized){
        Comb(MAXN);
        initialized=true;
    }
    ll res = frac[n] * infrac[n - k];
    res %= mod;
    return res;
}
}
using namespace Comb;

```

4.4 插板法 (stars and bars) 求非负整数解的数量

插板法 (stars and bars) 求非负整数解的数量
求解方程

$$x_1 + x_2 + \cdots + x_n = A$$

的解的个数 (A 为给定的正整数, n 为变量个数)。

先看好理解的情形: $x_i \geq 1$ 。把和 A 看成排成一行的 A 个小球, 相邻小球之间共有 $A-1$ 个空隙。我们要把这一排球切成 n 段, 第 i 段的球数就是 x_i 。因为每段至少有一个球 (对应 $x_i \geq 1$), 所以只需在这 $A-1$ 个空隙里插入 $n-1$ 块隔板、每个空隙至多插一块即可。于是方案数就是从 $A-1$ 个空隙里选 $n-1$ 个的组合数:

$$C_{A-1}^{n-1}$$

这一步非常直观, 所以插板法的“本体”其实就是 $x_i \geq 1$ 这个版本。

真正不好理解的是 $x_i \geq 0$ 。此时允许某一段为空, 没法直接套“在空隙里插板”的图像。不过我们可以做一个代数变形, 令

$$y_i = x_i + 1 \quad (i = 1, 2, \dots, n)$$

由 $x_i \geq 0$ 立刻得到 $y_i \geq 1$; 同时原方程两边各加上 n (共 n 个 $+1$), 右边变成 $A+n$:

$$y_1 + y_2 + \cdots + y_n = A + n$$

这样就把“ $x_i \geq 0$ 、和为 A ”的问题, **化归**成了刚刚已经解决的“ $y_i \geq 1$ 、和为 $A+n$ ”的问题。直接套用上面 $x_i \geq 1$ 的结论 (把那里的 A 换成 $A+n$) 即得:

$$C_{(A+n)-1}^{n-1} = C_{A+n-1}^{n-1} = C_{A+n-1}^A$$

4.5 Lucas 定理

Lucas 定理

注意, **如果 p 不是质数, 那么 Lucas 定理不成立**, 而且这个 **质数 p 还是要求比较小的, 一般就是 10^6 以内**。

对于质数 p 以及任意整数 $1 \leq m \leq n$, 有如下同余式成立:

$$C_n^m \equiv C_{n \bmod p}^{m \bmod p} \times C_{\lfloor n/p \rfloor}^{\lfloor m/p \rfloor} \pmod{p}$$

代码实现如下:

```

// Lucas 定理递归求解, 依赖于上方的组合数模板 CC
↪ 以及全局 mod
inline ll lucas(ll a, ll b) {
    // 如果 a, b 都小于 mod, 直接通过预处理的 CC
    ↪ 求解组合数
    if (a < mod && b < mod) return CC(a, b);
    // 否则, 根据 Lucas 定理递归求解
    return CC(a % mod, b % mod) * lucas(a / mod,
    ↪ b / mod) % mod;
}

```

注意, 在解决 Lucas 定理的时候, 余数最大为 $\text{mod}-1$, 所以只用预处理到 $\text{mod}-1$ 。**绝对不能预处理到 mod** , 否则 $\text{frac}[\text{mod}] = 0$, 会导致所有的 infrac 变成 0。

```

inline ll CC(ll n, ll k) {
    if (k < 0 || k > n) return 0;
    if(!initialized){
        // 【核心细节】: Lucas定理的余数最大为 mod-1,
        ↪ 所以只用预处理到 mod-1。
        // 绝对不能预处理到 mod, 否则 frac[mod] = 0,
        ↪ 会导致所有的 infrac 变成 0。
        Comb(mod - 1);
        initialized = true;
    }
    ll res = frac[n];
    res = (res * infrac[n - k]) % mod;
    res = (res * infrac[k]) % mod;
    return res;
}

```

4.6 组合相关公式以及技巧

4.6.1 离散变量概率变期望

当一个非负随机变量 X 只取整数值时, 其期望可以通过如下方式计算:

$$E[X] = \sum_{i=1}^{\infty} P(X \geq i)$$

即先对 $i = 1, 2, \dots$ 分别计算 $X \geq i$ 的概率, 将这些概率相加即为期望。

直觉上, 期望的定义是 $E[X] = \sum_{k=1}^{\infty} k \cdot P(X = k)$, 也就是每个取值 k 贡献了 k 份 $P(X = k)$ 。而 $P(X \geq i)$ 会把所有 $k \geq i$ 的概率加起来, 所以 $\sum_{i=1}^{\infty} P(X \geq i)$ 中, 取值 k 恰好被 $i = 1, 2, \dots, k$ 这 k 个求和项各计入一次, 合计贡献正好是 $k \cdot P(X = k)$, 与期望定义吻合。

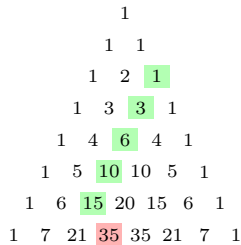
这个技巧在概率题中非常常用: 当直接求 $E[X]$ 困难时, 转而对每个阈值 i 求 $P(X \geq i)$ 往往更容易。

4.6.2 朱世杰恒等式 (Hockey-stick identity)

$$C_i^i + C_{i+1}^i + \cdots + C_n^i = C_{n+1}^{i+1}$$

等价形式：

$$\sum_{j=0}^r C_{i+j}^i = C_{i+r+1}^{i+1}$$



杨辉三角，第 0 行到第 7 行。曲棍球棒恒等式确认，例如：对于 $n=6, r=2$ ： $1+3+6+10+15=35$ 。

应用：当需要对连续上指标的组合数求和时（如 $\sum_{k=r}^n C_k^r$ ），可直接用朱世杰恒等式化为单个组合数 C_{n+1}^{r+1} ，将 $O(n)$ 的求和降为 $O(1)$ 。

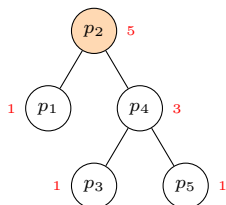
4.7 Hook-length 公式 (树版本)：递增标号计数

有根树 T (n 节点) 上把 $1, \dots, n$ 填到节点、要求每个非根节点标号 $>$ 父节点标号 (即整体是 min-heap / 递增树)，合法方案数

$$\#\{\text{递增标号}\} = \frac{n!}{\prod_{v \in T} \text{size}(v)}$$

节点 v 的「钩」就是它的整棵子树，钩长 $h(v) = \text{size}(v) = |T_v|$ 。

直观概率法证明。把 $1, \dots, n$ 均匀随机分派到节点上 ($n!$ 种等可能)。合法 min-heap $\iff$ 每个 v 是其子树 T_v 的最小标号；单事件概率 $= 1/\text{size}(v)$ ($\text{size}(v)$ 个位置地位对等)。这些事件相互独立——事件「 v 是 T_v 最小」只看 T_v 内部相对顺序，不同子树的相对顺序在均匀随机下互不影响。故合法概率 $\prod 1/\text{size}(v)$ ，方案数 $= n! \cdot \prod 1/\text{size}(v)$ 。■



图注： $p = [3, 1, 4, 2, 5]$ 的笛卡尔树；橙 = 根 (全局最小)；红字 = 子树 size。答案 $= 5! / (5 \cdot 1 \cdot 3 \cdot 1 \cdot 1) = 8$ 。

与笛卡尔树。若能从 a 唯一恢复 $\text{prev_smaller}/\text{next_smaller}$ (即唯一定出笛卡尔树形态)，则合法排列数 $= n! / \prod \text{size}_i$ ，其中 $\text{size}_i = \text{left}_i + \text{right}_i + 1$ 。例题：CF 2234E Vlad, Misha and Two Arrays。

拓展 1: 杨表 Hook-length (原版, Frame–Robinson–Thrall 1954)

分拆 $\lambda = (\lambda_1 \geq \lambda_2 \geq \dots) \vdash n$ 对应 Young 图；标准杨表 (SYT) 是把 $1, \dots, n$ 填进 Young 图使得每行严格递增、每列严格递增。SYT 计数

$$f^\lambda = \frac{n!}{\prod_{c \in \lambda} h(c)}, \quad h(c) = \underbrace{r(c)}_{\text{同行右侧}} + \underbrace{d(c)}_{\text{同列下方}} + 1.$$

本节的树版本就是这个公式在树上的类比 (把 Young 图换成树、 $h(c)$ 换成子树大小)。CP 里出现的场合：

- n 阶置换的 RSK 对应： $n! = \sum_{\lambda \vdash n} (f^\lambda)^2$ (每个 λ 贡献 $(f^\lambda)^2$ 对 (P, Q))。
- 长度 $\leq k$ 的最长上升子序列 = 第一行长度 $\leq k$ 的杨表；配合 hook-length 可数计数。
- 格路计数：从 $(0,0)$ 到 (a,b) 、不越对角线的路径数 $= f^{(b,a)}/\text{显式化简} = \text{Catalan} / \text{Ballot}$ 数 (Catalan 的另一种视角)。

拓展 2: 森林 / 有序 vs 无序 / q-类

- 森林版： n 节点森林 (k 棵根无序的树) 的递增标号数 $= n! / \prod_v \text{size}(v)$ ，跟单树公式一致 (等价于加一个虚根统一)。
- 有序 vs 无序树：本公式的树是「有孩子顺序」的 (如笛卡尔树左右子树有别)。若树是无序的 (同构算一次)，除以每个节点的孩子置换群阶 $\prod_v |\text{children}(v)|!$ 。
- q-类版：对树的降序列 (descent) 生成函数 $\sum_T q^{\text{maj}(T)} = [n]_q! / \prod [\text{size}(v)]_q$ ，其中 $[m]_q = (1 - q^m)/(1 - q)$ 。CP 少用但组合数学味浓，遇到“q-类比” / “maj 生成函数” 题目时可想。
- 路径 / 链退化：若树是一条链 (n 个节点从上到下)， $\text{size} = n, n-1, \dots, 1$ ，公式给 $n!/n! = 1$ ——链上只有唯一递增填法，符合直觉。
- 完全二叉树： $n = 2^h - 1$ 层完全二叉树， $\prod \text{size}$ 有闭式 $\prod_{k=0}^{h-1} (2^{h-k} - 1)^{2^k}$ ，可直接查表验证。

4.8 Erdős–Szekeres 定理 (单调子序列)

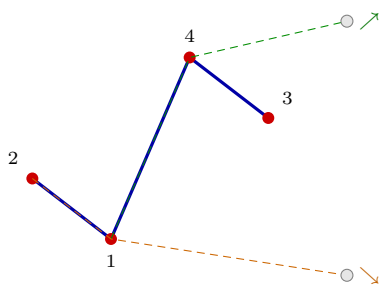
任意 n 个互不相同的实数构成的序列，只要长度

$$n \geq (r-1)(s-1) + 1,$$

就必含一个长度为 r 的 (严格) 递增子序列，或一个长度为 s 的 (严格) 递减子序列。

取 $r=s=3$ ：长度 $\geq (3-1)(3-1) + 1 = 5$ 的序列必含长度 3 的单调子序列；反过来，不含长度 3 单调子序列的序列最长只有 4，临界长度恰好是 5。

鸽巢证明：给每个位置记一对数 (u, v) —— u 是以它结尾的最长 (非严格) 递增子序列长度， v 是最长递减长度。若全程不出现长度 3 的单调子序列，则 $u, v \in \{1, 2\}$ ，组合只有 $2 \times 2 = 4$ 种；第 5 个位置必与前面某位置的 (u, v) 相同，而两位置标签相同会强制其中一维 $+1$ ，矛盾。故最长为 4。



图注：蓝 = 锯齿序列 2143（长度 4，任取 3 个都不构成非严格单调，合法）；补第 5 个点（灰）无论放高放低，都与前两点凑出长度 3 单调子序列（绿 $1 \leq 4 \leq 4.6$ 递增 / 橙 $2 \geq 1 \geq 0.4$ 递减），故临界长度为 5。

应用：统计「不含长度 3 非严格单调子序列」的连续子段个数时，由本定理合法子段长度 ≤ 4 ，对每个右端点向左只需枚举长度 $1 \sim 4$ 暴力判定， $O(n)$ 完成。
例题：[牛客河南萌新联赛 2025（一）M 无聊的子序列](#) ($n \leq 10^6$)。该题是**非严格版**（允许相等），合法子段长度同样 ≤ 4 ；定理原版针对严格单调（元素互异），可与 Dilworth 定理、最长上升子序列对照理解。

5 博弈论

5.1 Nim 博弈

规则：桌上有 n 堆石子，第 i 堆有 $a_i \geq 0$ 颗。两人轮流操作，每回合选一堆非空的石子，从中拿走任意正整数颗（至少 1 颗，不能跨堆）。无法操作（所有堆都空）的玩家落败。问先手能否取胜。

结论（Bouton 定理）：先手胜当且仅当

$$a_1 \oplus a_2 \oplus \cdots \oplus a_n \neq 0.$$

把这个异或值称为局面的 **Nim 和** (Nim-sum)。Nim 和为零的局面是 **P 局面** (Previous, 前一手获胜，即当前轮到的人必败)；非零是 **N 局面** (Next, 当前轮到的人必胜)。

为什么是异或？三段闭包性：

1. **终止 = P**：全 0 局面 Nim 和为 0，确实无路可走、当前人输。
2. **P 走完必入 N** (封死 P 出口)：从 Nim 和 = 0 出发，任何合法操作只动一堆且必使该堆严格变小，单堆变化必然改变异或——**改一堆就一定让 Nim 和从 0 变非 0**。所以 P 局面下你怎么走都把回合白送给对手。
3. **N 总能走回 P** (构造 N 通道)：从 Nim 和 $s \neq 0$ 出发，找 s 的最高位 k 。 s 在第 k 位为 1 意味着 a_i 中至少有一个在第 k 位为 1，挑这样的一堆 a_j ，把它改为 $a_j \oplus s$ 。两个关键事实：(i) a_j 第 k 位被 s 翻成 0、更高位不动，所以 $a_j \oplus s < a_j$ ，是合法的“减一个正数”操作；(ii) 操作后总异或变成 $s \oplus s = 0$ 。因此 N 局面总能一步把 P 局面“踢回”对手。

三段闭合后再做归纳：终止 P 局面下手必败 $\Rightarrow$ 一切 P 局面下手必败（因为只能进入 N，再被对手踢回 P 直至终止）；一切 N 局面下手必胜（一步进入 P，把上述循环转嫁给对手）。

Nim 走法（构造性）：判负只是判定，但 **遇到 N 局面时必须知道“该动哪一堆、动到几”**。算法照搬上面 (3)：

```
// 返回 (堆下标 j, 改后的新值); 保证 a_j 严格减小
pair<int, ll> nim_move(const vector<ll>& a) {
    ll s = 0;
    for (ll x : a) s ^= x;
    if (s == 0) return {-1, -1}; // P 局面,
    ↪ 无必胜走法
    for (int j = 0; j < (int) a.size(); ++j) {
        ll t = a[j] ^ s;
        if (t < a[j]) return {j, t}; //
        ↪ 找到含最高位的那一堆
    }
    return {-1, -1}; // 不会到这
}
```

注意“含 s 最高位”的判定可以不显式取最高位：直接看 $a_j \oplus s < a_j$ 是否成立——等价于 a_j 在 s 的最高位上为 1。这个写法少踩位运算坑。

推广：Sprague-Grundy 定理。对任意公平组合游戏（双方走法对称、有限步必终止、无随机），定义局面 u 的 **SG 值** $SG(u) = \text{mex}\{SG(v) : u \rightarrow v\}$ 。则：

- 单局游戏 u 是 P 局面 $\iff SG(u) = 0$ 。
- 多个独立游戏并行进行（每回合任选一局走一步），整体 SG 值 $= \bigoplus_i SG(u_i)$ 。

Nim 单堆的 SG 值就是堆里石子数（堆 a 一步可达 $\{0, 1, \dots, a-1\}$, mex 即 a ），所以多堆 Nim 的“异或为零判 P”正是 SG 定理在 Nim 上的特化。**遇到看似复杂的博弈，先尝试拆成若干独立子游戏 + 求每个 SG + 异或起来**——下一节的“三角形博弈”就是把“非标准的单游戏”通过换元映回标准 Nim 的最简案例。

5.1.1 三角形博弈（带约束的 Nim）

给定能构成非退化三角形的三整数 a, b, c （即满足 $a + b > c, b + c > a, a + c > b$ ）。两人轮流操作，每回合选一个数减去任意正整数；操作后若三数无法构成非退化三角形，则该玩家落败。先手能否获胜？

结论：先手胜当且仅当

$$(a-1) \oplus (b-1) \oplus (c-1) \neq 0.$$

核心观察——把“边长”换元为“石子堆”：令 $x_i = s_i - 1$ (s_i 是边长)。每条边至少为 1，对应每堆石子至少为 0；“减去任意正整数”在两套语言下完全一致。终止局面 $(1, 1, 1)$ 对应 $(0, 0, 0)$ ，与标准 Nim 的零局面重合。**换言之，剥掉三角约束之后，这就是三堆 Nim**。剩下要论证的只是：三角约束不会挡掉从 N 局面走向 P 局面的那一步（沿用上一节的 Nim 走法）。

为什么三角约束不挡 Nim 走法：从异或非零的 (a, b, c) 出发，设 $s = (a-1) \oplus (b-1) \oplus (c-1)$ ，找到 $(a-1, b-1, c-1)$ 中含 s 最高位的那一堆，标准 Nim 操作把它改成“它 $\oplus s$ ”——这是一次严格变小的修改。不妨设被改的是 $c-1$ ，新值 $c'-1 = (c-1) \oplus s = (a-1) \oplus (b-1)$ 。此时 $|a-b| \leq c'-1 \leq a+b-2 < a+b$ ，新三角不等式自动成立（最长边假设依然成立时同理），所以“修复异或”的 Nim 走法始终是合法着。

反向也封死：从异或为零的 P 局面出发，任何合法操作只动一堆，异或必然变成非零；对手回到 N 局面继续兑现获胜策略。

实战提示：实现一行就够——读入三个数后输出 $((a-1) \oplus (b-1) \oplus (c-1)) ? \text{"Win"} : \text{"Lose"}$ 。范例题：QOJ 4545 Triangle Game (凯特和艾米丽科， $1 \leq a, b, c \leq 10^9, T \leq 10^4, O(1)$ 一次输出)。推广视角：所有“看似有附加约束的 Nim”都值得先做这套换元——**把约束的下界吸收到偏移里、把“零状态”对齐成 $(0, 0, \dots)$ 、再核对“最优 Nim 走法是否被约束挡掉”，三步通吃**；本题正是 -1 偏移加“最长边可调”的最简版本。

5.1.2 Moore's Nim-K (每回合可同时动 K 堆)

规则: n 堆石子, 每回合至多选 K 堆, 每选中的堆都取走任意正整数颗 (不同堆取数可以不同), 取到最后的一颗者胜。 $K = 1$ 即标准 Nim。

结论: 把每堆数写成二进制, 记第 i 位上 1 的个数为 ones_i 。先手败 $\iff$ 每个 ones_i 都是 $(K+1)$ 的倍数。 $K = 1$ 退化成“每位 ones_i 都是偶数”, 即 XOR = 0。

直觉: 标准 Nim 的“模 2 异或”是 $K = 1$ 的特例, 因为每回合只能动 1 堆, 每一位上的 1 个数变化量 $\in \{-1, 0, +1\}$, 对手可恢复奇偶。 Nim-K 下变化量 $\in \{-K, \dots, K\}$ (每选中堆贡献 ± 1), 对手可恢复“模 $K+1$ ”残差。

5.1.3 反 Nim (Anti-Nim / Misère Nim)

规则: 与标准 Nim 完全相同, 但取到最后的一颗者输 (拿到最后一颗的“出局”)。

结论: 先手胜 $\iff$ 满足下列其一 ——

- 所有堆都 ≤ 1 且 Nim 和 = 0 (等价于“全是 1 且堆数为偶数”);
- 至少一堆 > 1 且 Nim 和 $\neq 0$ 。

直觉: 当所有堆都 ≤ 1 时游戏退化“轮流取走单颗”, 胜负由堆数奇偶决定 (misère 下偶数堆先手胜)。当存在 > 1 的“大堆”时, 先手始终能控制把局面引入“全 1 局面, 且堆数对自己有利”——具体策略类似标准 Nim, 只是终态判定反过来。“ > 1 堆 + Nim 和非零”是先手能掌控终态奇偶的标志。

5.1.4 阶梯 Nim

规则: n 级台阶 (编号 $1 \dots n$), 第 i 级有 a_i 颗石子。每回合从某级 $i \geq 1$ 拿任意正数颗放到第 $i-1$ 级 (第 1 级的石子被放到地面 0 级, 地面石子不能再动), 取走最后一颗者胜 (即让对手面对“无可动石子”局面者胜)。

结论: 只看奇数级台阶上的石子做标准 Nim——先手败 $\iff a_1 \oplus a_3 \oplus a_5 \oplus \dots = 0$ 。

直觉——“偶数级是镜子”: 偶数级上的动作可以被对手原样模仿——你从第 $2k$ 级搬 x 颗到 $2k-1$ 级, 对手就从 $2k-1$ 级搬同样 x 颗到 $2k-2$ 级, 把你“白送”到偶数级的石子继续往下推, 整体偶数级局面回到等价状态。所以偶数级上的动作不影响胜负, 等价 Nim 只发生在奇数级。

5.1.5 Lasker's Nim (可拆分的 Multi-SG)

规则: n 堆, 每回合二选一——(a) 从某堆取若干颗 (标准 Nim 操作); (b) 把某堆 ≥ 2 颗的拆成两堆非空。取到最后的一颗者胜。

结论: 单堆 SG 函数为

$$\text{SG}(x) = \begin{cases} x-1, & x \bmod 4 = 0 \\ x, & x \bmod 4 \in \{1, 2\} \\ x+1, & x \bmod 4 = 3 \end{cases}$$

多堆答案 = $\bigoplus_i \text{SG}(a_i)$, 非零时先手胜。

打表速记: $x = 0, 1, 2, 3, 4, 5, 6, 7$ 时 SG 为 $0, 1, 2, 4, 3, 5, 6, 8$ ——每 4 个一组, 组内除了 $4k$ 与 $4k+3$ 互换以外都是恒等。背不住时直接用通用 SG 暴力打表 (见下节) 现场跑。

5.2 巴什博弈 (单堆有上限取)

规则: 单堆 N 颗石子, 每回合取 $1 \sim M$ 颗, 取到最后的一颗者胜。

结论: 先手败 $\iff N \bmod (M+1) = 0$ 。

为什么是 $M+1$ 周期——“凑齐 $M+1$ ”的控制术: 双方可以稳定地把每一回合的总取数控制为 $M+1$ (一方取 $X \in [1, M]$, 另一方立即取 $M+1-X$, 二者必然合法)。所以 N 被 $M+1$ 整除的局面是 P 局面: 当前人无论怎么取, 对手都能“补齐成 $M+1$ ”, 让 N 严格按 $M+1$ 递减直到 0 (最后一颗永远被对手拿走)。 N 不被整除时, 先手取走余数 $R = N \bmod (M+1)$ 把局面调成整除留给对手, 化身“控盘者”反将一军。

扩展巴什博弈: 每回合取走 $X \in [a, b]$ 颗, 剩余不足 a 时不能再取, 取到最后的一颗者胜 (注意: 剩余 $R < a$ 时游戏自然结束, “最后一颗”指 R 中是否含取数边界——按惯例, 无法再取的当前玩家算作“未取到最后一颗”, 即输家)。

结论: 以 $a+b$ 为周期划分, 先手败 $\iff N \bmod (a+b) < a$ 。

$a+b$ 周期的直觉: 把上面“凑齐 $M+1$ ”换成“凑齐 $a+b$ ”——一方取 $X \in [a, b]$, 对手取 $a+b-X \in [a, b]$ 始终合法。余数 $R \in [0, a)$ 是 P 局面: 先手取走 $X \geq a$ 后剩下 $N-X$, 对手补成 $a+b$ 周期把局面继续锁在余数 $< a$ 区段, 最终把 $< a$ 的“垃圾尾巴”留给先手取不动。余数 $R \in [a, b]$ 时先手直取 R ; 余数 $R \in (b, a+b)$ 时先手取 $X = R - (a-1) \in (b-a+1, b]$ (边界容易验证), 把局面调到余数 $a-1$ 留给对手。

5.3 SG 函数的暴力打表与 Anti / Every 变体

通用 SG 暴力 (适合状态数 $\leq 10^6$ 的题目, 没法找闭式时打表观察规律):

```
// 单堆从 x 颗一步可达的下一状态集合由 trans(x)
// 给出
// SG(x) = mex{ SG(y) : y in trans(x) }
int sg[N];
void compute_sg(int upTo) {
    for (int x = 0; x <= upTo; ++x) {
        vector<int> nxt = trans(x);
        vector<bool> seen(nxt.size() + 1, false);
        for (int y : nxt) if (sg[y] < (int) seen.size())
            seen[sg[y]] = true;
        int m = 0; while (m < (int) seen.size() && seen[m]) ++m;
        sg[x] = m;
    }
}
```

用法: 把所有子游戏的 SG 异或起来; 非零先手胜。打表观察规律是博弈题最常用的找通项手段——先暴力

跑出 $SG(0 \dots 100)$ 看是否周期 / 等于 x / 等于 $\lfloor x/k \rfloor$ 之类的简单形式，再去证。

5.3.1 反 SG (Anti-SG)

规则：与 SG 游戏一致，但无法行动者胜（最先不能动者赢）。

结论（与反 Nim 同形）：先手胜 $\iff$ 满足其一——

- 所有子游戏 $SG \leq 1$ 且总 $XOR = 0$;
- 至少一个子游戏 $SG > 1$ 且总 $XOR \neq 0$ 。

5.3.2 Every-SG (每回合走”所有还能动的石子”)

规则：DAG 上若干石子，每回合必须把所有还能动的石子各走一步（不能选择不动），无法行动者输。

结论：对每个起点求 $step(v)$ ——在该子游戏中博弈能持续的回合数（必胜方尽量缩短、必败方尽量拖长）：

- 终止节点： $step = 0$ 。
- $SG(v) > 0$ (必胜)： $step(v) = 1 + \min\{step(w) : v \rightarrow w, SG(w) = 0\}$ 。
- $SG(v) = 0$ (必败)： $step(v) = 1 + \max\{step(w) : v \rightarrow w\}$ 。

先手胜 $\iff$ 各子游戏 $step$ 的最大值是奇数。 直觉：所有子游戏并行推进，最长那局决定整盘何时结束；该局轮到谁动谁就输，奇偶反转后对应先手胜。

5.3.3 实战范例：CCPC 2025 网络赛 F「连线博弈」——XOR 哈希切子游戏

题意（第十一届 CCPC 网络预选赛 F）：单位圆周上等距 n 个点 ($2 \leq n \leq 10^9$)，初始有 m 条两两严格不交且不共端点的弦 ($0 \leq m \leq 10^5, \sum m \leq 10^5$)。Alice/Bob 轮流操作，每回合选两个未被任何弦端点占用的”自由点”画一条不与已有弦相交的新弦，无法操作者输。问 Alice 是否必胜。

子游戏分解：每条弦把圆盘切成两半，自由点被弦端点序列划分成若干独立子游戏。 x 个自由点的子游戏一步操作占掉 2 个点，并把段切成大小为 a, b ($a + b = x - 2$) 的两个新子游戏。故

$$SG(x) = \text{mex}_{a+b=x-2} (SG(a) \oplus SG(b)), \quad SG(0) = SG(1) = 0.$$

暴力打表至 500 后发现 $x \geq 36$ 起呈周期 34，但 **$idx = 52$ 是异常项** ($sg[52] = 2$ ，但 $x \geq 70$ 落到 $idx = 52$ 的真实周期值是 $sg[86] = 9$)，**必须特判**，否则会被 hack 数据 $n = 8, m = 2, (0, 1)(4, 5)$ 卡掉（朴素周期答 NO，正解 YES）。**教训**：找到看似规整的周期务必本地用暴力对拍校验首尾几百项，不要看一眼”前几行循环对了”就上交。

XOR 哈希切子游戏（关键技巧）：题面要求”识别哪些自由点共属一个子游戏”，传统做法（按弦端点排序，相邻端点之间一段算一个子游戏）会把”两条互不相交的弦同时切到的中间区段”误判为多个独立段。**用随机 XOR 标记代替几何判定**——给每条弦 (x, y) 生成随机权 r ，记 $mp[x] \oplus r, mp[y] \oplus r$ 。按下标升序扫

描端点，维护前缀异或 cur ； cur 相同的下标段就属于同一子游戏（因为弦的两个端点同时翻转 cur 两次，段内自由点的 cur 等于”包住它的所有未闭合弦”的随机权异或，几何上”被同一组弦包裹”等价于”在同一子游戏”）。最后对每个 cur 桶累加自由点数，再异或所有 SG(桶大小)。

```
// 预处理 SG 至 500
sg[0] = sg[1] = 0;
for (int i = 2; i <= 500; ++i) {
    set<int> s;
    for (int j = 0; j <= i - 2; ++j)
        s.insert(sg[j] ^ sg[i - 2 - j]);
    int mex = 0; while (s.count(mex)) ++mex;
    sg[i] = mex;
}

// 周期 34 (从 i=36 起), idx=52 异常项特判
auto SG = [&](ll x) -> int {
    if (x <= 200) return sg[x];
    int idx = 36 + (x - 36) % 34;
    return idx == 52 ? sg[idx + 34] : sg[idx];
};

// 单组询问: XOR 哈希 + 扫描分桶
void solve() {
    cin >> n >> m;
    map<ll, ll> mp; // 端点 -> 该点累计的随机权异或
    mt19937_64 rng(time(0));
    for (int i = 0; i < m; ++i) {
        ll x, y; cin >> x >> y;
        ll r = rng();
        mp[x] ^= r; mp[y] ^= r; // 弦两端各异或同一个 r
    }

    if (m == 0) { cout << (SG(n) ? "YES\n" : "NO\n"); return; }

    map<ll, ll> bucket; // cur -> 该组子游戏的自由点总数
    ll up = -1, cur = 0;
    mp[n] = 0; // 末尾哨兵: 把最大端点之后到 n-1 的尾段也结算
    for (auto& [u, v] : mp) {
        bucket[cur] += max(0LL, u - up - 1); // [up+1, u-1] 的自由点数
        up = u; cur ^= v;
    }

    ll ans = 0;
    for (auto& [_, sz] : bucket) ans ^= SG(sz);
    cout << (ans ? "YES\n" : "NO\n");
}
```

为什么 XOR 哈希这一招通用：本质是用差分维护”被哪些区间覆盖”——每条弦 (x, y) 的”覆盖范围”在排序后的端点序列上是一段，而 $mp[x] \oplus r; mp[y] \oplus r$ ；正是**异或差分** (cur 在 x 之后翻入 r ，在 y 之后翻出 r ， $[x, y)$ 段的 cur 多含 r)。任何”按区间覆盖签名分组”的题都可套用这套：把”判断两个位置是否属于同一组”翻译成”它们的 cur 哈希是否相等”，免掉显式区间扫线 / 并查集合并。**相关题**：HDU/CodeForces

上”求被不同区间集合覆盖的位置数 / 等价类数”系列均可一键套用。

5.4 威佐夫博弈 (双堆联动)

规则: 两堆石子 (a, b) (设 $a \leq b$)。每回合二选一——(a) 从单堆取任意正数颗; (b) 从两堆同时取走相同的正数颗。取到最后一颗者胜。

结论: 奇异 (P) 局面是 $(a_k, b_k) = ([k\varphi], [k\varphi] + k)$, $k = 0, 1, 2, \dots$, 其中 $\varphi = \frac{1+\sqrt{5}}{2}$ 是黄金分割比。即 $(0, 0), (1, 2), (3, 5), (4, 7), (6, 10), (8, 13), \dots$ 。当前 (a, b) 是 P 局面 $\iff \lfloor (b-a) \cdot \varphi \rfloor = a$ 。

Beatty 定理背景: 两序列 $\{[k\varphi]\}$ 与 $\{[k\varphi^2]\}$ 由 Beatty 定理给出“ $\mathbb{N}^+$ 的不交划分”, 恰好把每个正整数唯一指派为某个 a_k 或 $b_k = a_k + k$ 。这保证了奇异局面集合”覆盖且只覆盖一次”。

大数精度警告: $N \leq 10^9$ 时 double 误差刚好够呛, 建议直接用高精度常数 $\varphi \approx 1.6180339887498948482$, 或直接打表黄金比的两个 Beatty 序列 (项数 $\leq 10^9 / \varphi \approx 6 \times 10^8$ 不行, 所以现场公式判定):

```
const double PHI = 1.6180339887498948482L;
bool first_loses(long long a, long long b) {
    if (a > b) swap(a, b);
    return (long long)((b - a) * PHI) == a;
}
```

5.5 斐波那契博弈 (前手两倍约束)

规则: 单堆 N 颗。先手第 1 回合可取 $1 \sim N-1$ 颗 (不能一次拿光), 此后每回合的取数上限为上一回合所取数的 2 倍。取到最后一颗者胜。

结论: 先手败 $\iff N$ 是斐波那契数 ($N \in \{1, 2, 3, 5, 8, 13, 21, \dots\}$)。

直觉——齐肯多夫 (Zeckendorf) 分解: 每个非斐波那契数都能唯一拆成若干不相邻斐波那契数之和 $N = F_{k_1} + F_{k_2} + \dots + F_{k_t}$ ($k_1 > k_2 > \dots > k_t$, $k_i - k_{i+1} \geq 2$)。先手取走最小项 F_{k_t} , 由 Fibonacci 递推 $F_{k_{t-1}} > 2F_{k_t}$, 对手无法一次拿完下一项 $F_{k_{t-1}}$ 段, 必然被迫”破坏”它, 先手再取该段最小项继续逼迫……层层剥到只剩 F_{k_1} , 此时正好对手取一颗、先手取一颗交替到底。 N 本身是斐波那契数时分解只有一项, 先手第一手就被”取数 $\leq N-1$ 不能一次拿完 + 任取必让对手反吃 $\leq 2X$ 区间”的双重约束封死。

```
bool first_loses_fib(long long n) {
    long long a = 1, b = 2;
    while (b <= n) { long long c = a + b; a = b; b = c; }
    // 此时 a 是 <= n 的最大 Fibonacci
    return a == n;
}
```

5.6 图上删边游戏

5.6.1 树上删边 (Colon Principle)

规则: 有根树, 每回合删一条边, 与根失去连接的部分整体被删除 (即删去子树)。无边可删者输。

SG 公式 (Colon Principle, Smith 定理): 以节点 v 为根的子树 SG 值 $g(v)$ 满足

$$g(\text{叶子}) = 0, \quad g(v) = \bigoplus_{c \text{ 是 } v \text{ 的孩子}} (g(c) + 1).$$

注意 $+1$ 是普通整数加法 (含义: ”孩子子树的 SG 等价为一堆 $g(c)$ 颗石子, 挂上连父边后再加一颗”), 异或才是组合多个独立分支。

先手胜 $\iff g(\text{根}) \neq 0$ (与 SG 通则一致)。

```
// 返回以 u 为根的子树 SG 值
function<int(int, int)> dfs = [&](int u, int fa) -> int {
    int s = 0;
    for (int v : adj[u]) if (v != fa) s ^= (dfs(v, u) + 1);
    return s;
};
cout << (dfs(root, -1) != 0 ? "Win\n" : "Lose\n");
```

5.6.2 无向图删边 (Fusion Principle)

规则: 连通无向图 (可含环、重边、自环), 指定一个根, 每回合删一条边, 与根失联的部分整体删除。无边可删者输。

Fusion Principle (环约简 $\rightarrow$ 化为树上删边):

- **奇环** (含奇数条边的环, 含自环): 缩成一个新点 + 一条挂在它上面的悬空边。
- **偶环**: 缩成一个新点 (不留多余边)。
- 原本连接到环上任意点的所有边, 全部接到这个新缩出来的点上。

反复约简到没有环为止, 剩下的就是一棵树, 套用上一节的 Colon Principle 求 SG。

直觉: 环上任意一条边被删后剩下一条路径——”环作为整体”的 SG 等于”等价路径的 SG”, 奇数 / 偶数环的等价物正好是”1 条边” / ”0 条边” (这是 Hackenbush 经典结论)。所以缩点保留这条等价残留即可。

6 进阶数学

6.1 FFT 多项式乘法 (非递归)

采用非递归版本实现 FFT (蝴蝶变换), 能显著减小常数并优化内存使用。这里通过 multiply 演示了如何运用 fft 对两个多项式 (系数数组表示) 进行乘法操作。

```
// 理论讲解
↪ https://www.bilibili.com/video/BV1za411F76U/
// 参考 (抄的)
↪ https://cp-algorithms.com/algebra/fft.html
// 多项式乘法
↪ https://www.luogu.com.cn/record/229525769
using CD = complex<double>;

void fft(vector<CD> &a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <= 1) {
        double ang = 2 * pi / len * (invert ? -1 : 1);
        CD wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            CD w(1);
            for (int j = 0; j < len / 2; j++) {
                CD u = a[i + j], v = a[i + j + len / 2];
                ↪ * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= wlen;
            }
        }
    }
    if (invert) {
        for (CD &x : a) x /= n;
    }
}

vector<ll> multiply(vector<ll> &a, vector<ll> &b) {
    ↪ &b) {
        vector<CD> fa(a.begin(), a.end()),
        ↪ fb(b.begin(), b.end());
        int n = 1;
        while (n < (int)(a.size() + b.size())) {
            n <= 1;
        }
        fa.resize(n); fb.resize(n);
        fft(fa, false); fft(fb, false);
        for (int i = 0; i < n; i++) fa[i] *= fb[i];
        fft(fa, true);
        vector<ll> res(n);
        // 避免精度损失
        for (int i = 0; i < n; i++) res[i] =
        ↪ llround(fa[i].real());
        return res;
    }
}
```

6.1.1 高精度乘法 (基于 FFT)

<https://www.luogu.com.cn/record/229526218>

大整数乘法的核心思想: 将大整数的每一位看作多项式的一个系数。例如 1234 可以表示为多项式 $4 + 3x + 2x^2 + x^3$ (低位在前)。两个大整数相乘等价于两个多项式相乘, 乘完后对结果数组统一处理进位。

```
inline void __() {
    string A, B;
    cin >> A >> B;
    vector<ll> a, b;
    for (int i = (int)A.size() - 1; i >= 0; --i) {
        a.push_back(A[i] - '0');
    }
    for (int i = (int)B.size() - 1; i >= 0; --i) {
        b.push_back(B[i] - '0');
    }

    vector<ll> ans = multiply(a, b);
    ll carry = 0;
    for (int i = 0; i < (int)ans.size(); ++i) {
        ans[i] += carry;
        carry = ans[i] / 10;
        ans[i] %= 10;
    }
    while (!ans.empty() && ans.back() == 0) {
        ans.pop_back();
    }
    if (ans.empty()) {
        cout << 0;
        return;
    }
    for (int i = (int)ans.size() - 1; i >= 0; --i) {
        cout << ans[i];
    }
}
```

6.1.2 FFT 的使用方法与常见应用

multiply(a, b) 的本质是: 给定多项式 $A(x) = \sum a_i x^i$ 和 $B(x) = \sum b_i x^i$, 返回乘积多项式 $C(x) = A(x) \cdot B(x)$ 的系数数组, 其中 $c_k = \sum_{i+j=k} a_i \cdot b_j$ 。因此, 只要问题能转化为这种卷积形式, 就能用 FFT 加速到 $O(n \log n)$ 。

1. 加法卷积计数

最典型的场景: 有两组数, 问它们各取一个数相加能得到每个和的方案数。

例如掷两颗骰子, 求每种点数之和的方案数。将骰子 A 的各面出现次数存入 a[] (a[k] 表示点数 k 出现的次数), 骰子 B 同理存入 b[], 则 multiply(a, b) 的结果 c[k] 就是和为 k 的方案数。

```
// 例: 两个数组各取一个数, 统计每种和的方案数
// a[i] = 数 i 在第一组中出现的次数
// b[j] = 数 j 在第二组中出现的次数
// c[k] = 和为 k 的方案数 = sum(a[i]*b[k-i])
vector<ll> c = multiply(a, b);
```

2. 字符串匹配 (含通配符)

给定文本串 T (长度 n) 和模式串 P (长度 m), 对于 T 的每个起始位置 t , 定义失配度:

$$D(t) = \sum_{j=0}^{m-1} (T_{t+j} - P_j)^2 \cdot w_j$$

其中 w_j 是通配符权重 (通配符处 $w_j = 0$)。 $D(t) = 0$ 表示位置 t 完全匹配。

将 P 翻转为 $P'_j = P_{m-1-j}$, 展开后各项都可化为卷积形式, 分别用 FFT 计算后合并。

```
// 无通配符的简单情形: 检查  $T$  和  $P$   
↪ 是否在每个位置匹配  
// 构造  $a[i] = T[i]$ ,  $b[j] = P[m-1-j]$  (翻转  $P$ )  
// 则  $c[t+m-1] = \text{sum}(T[t+j] * P[m-1-j])$  即为位置  
↪  $t$  的内积  
// 更一般地, 含通配符时需要构造多组卷积分别计算  
vector<ll> a(n), b(m);  
for (int i = 0; i < n; i++) a[i] = T[i] - 'a' +  
↪ 1;  
for (int j = 0; j < m; j++) b[j] = P[m - 1 - j]  
↪ - 'a' + 1;  
vector<ll> c = multiply(a, b);  
//  $c[t + m - 1]$  即为位置  $t$  的匹配内积
```

3. 高精度乘法

如上一节所述, 将大整数每一位作为多项式系数, 调用 multiply 后统一处理进位。

4. 多项式求值与插值的中间步骤

分治 FFT 可以加速多项式多点求值、多点插值等操作, 核心都是递归地调用 multiply 合并子问题的结果多项式。

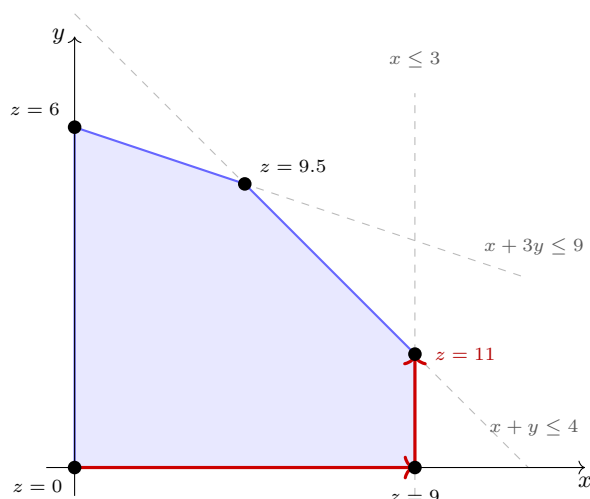
6.2 线性规划（两阶段单纯形 Simplex）

求解标准型

$$\max c^T z \quad \text{s.t.} \quad Az \leq b, z \geq 0.$$

三样输入分别是： A 是约束系数（ A_{ij} = 第 i 条约束里变量 j 的系数）， b 是每条约束的上限， c 是要最大化的那个式子的系数。代码里就直接叫 `coef` / `limit` / `profit`，解向量叫 x 。

可行域是若干半空间交出的凸多面体，最优值必在某个顶点处取到。单纯形法就是从一个顶点出发，每次沿一条棱走到相邻的更优顶点（一次 `pivot` = 一个非基变量入基、一个基变量出基），直到目标行里没有改进方向为止。



图中目标为 $\max z = 3x + 2y$ ，路径 $(0,0) \rightarrow (3,0) \rightarrow (3,1)$ 即两次 `pivot` 后到达最优顶点。

实现要点：

- **字典表连续存储**： $(rows + 2) \times (cols + 2)$ 一维数组行主序，比 `vector<vector<double>>` 少一层间接寻址，内层循环无分支可向量化。
- **两阶段**： b 有负分量时原点不可行，先引入人工变量 x_0 跑第一阶段 $\min x_0$ ；若最优 $x_0 > 0$ 则原问题不可行。 $b \geq 0$ 时直接跳过第一阶段。
- **Devex 定价**：入基列挑 d_j^2/w_j 最大者（ d_j 为目标行系数、 w_j 为参考权重），比朴素 Dantzig 规则（挑最负的 d_j ）迭代次数少很多。
- **防循环**：退化时目标值长期不推进，连续 `stallLimit` 次后切 Bland 最小下标规则（保证有限步终止），推进后自动切回 Devex。
- **出基行**：最小比值 b_i/a_{is} ；比值持平时选主元绝对值更大的一行，数值更稳。

返回值：最优值；无界返回 $+\infty$ ，不可行返回 $-\infty$ ；`solution` 非空时回填一组最优解。

建模转换：等式约束拆成 $\leq$ 与 $\geq$ 两条； $\geq$ 约束两边取负变 $\leq$ ；`min` 问题把 `profit` 整体取负、答案再取负；变量可正可负时拆成 $z = z^+ - z^-$ 两个非负变量。

```
// 两个变量 x, y, 都 >= 0; 要最大化 3x + 2y
// 三条约束: x + y <= 4, x + 3y <= 9, x <= 3
vector<vector<double>> coef = {{1, 1}, // 第 1 条约束里 x, y 的系数
                              {1, 3}, // 第 2 条
                              {1, 0}}; // 第 3 条 (y 不出现, 系数写 0)
vector<double> limit = {4, 9, 3}; // 每条约束不等号右边的上限
vector<double> profit = {3, 2}; // 目标 3x + 2y 里 x, y 的系数
SimplexLP lp;
lp.init(coef, limit, profit);
vector<double> x; // 取到最优时每个变量的值
double best = lp.solve(&x); // best = 11, x = {3, 1}
```

```

// 变量  $x[0..cols]$  全都  $\geq 0$ ; 每条约束形如  $coef[i][0]*x[0] + \dots \leq limit[i]$ ;
// 目标是让  $profit[0]*x[0] + profit[1]*x[1] + \dots$  尽量大。
struct SimplexLP {
    // 核心就是一张表  $tab$ ,  $(rows + 2)$  行  $x$  ( $cols + 2$ ) 列, 拍平成一维存 (比二维  $vector$  少一层寻址):
    // 每行一条约束, 每列一个变量, 最后一列放这条约束还剩多少余量
    // 多出的倒数第 2 行 = 目标 (每个变量再加一单位能赚多少), 最后一行只在 " 先找可行点 " 时用
    // 多出的倒数第 2 列 = 辅助变量  $x_0$ , 同样只在 " 先找可行点 " 时用
    int rows = 0; // 约束条数
    int cols = 0; // 变量个数
    int stride = 0; // 一行占多少个  $double$  ( $= cols + 2$ )
    vector<double> tab;
    vector<int> basis; //  $basis[i]$  = 第  $i$  行当前解出来的那个变量的编号
    vector<int> nonbasis; //  $nonbasis[j]$  = 第  $j$  列对应的变量编号, 这些变量当前取 0
    double eps = 1e-9;
    long long pivots = 0; // 换基次数, 调试看规模用
    vector<double> weight; // DeveX 参考权重, 挑变量时用

    double* row(int i) { return tab.data() + static_cast<size_t>(i) * stride; }
    const double* row(int i) const { return tab.data() + static_cast<size_t>(i) * stride; }

    //  $coef[i][j]$ : 第  $i$  条约束里变量  $j$  的系数  $limit[i]$ : 第  $i$  条约束的上限
    //  $profit[j]$ : 变量  $j$  每多一单位, 目标增加多少
    void init(const vector<vector<double>>& coef,
              const vector<double>& limit,
              const vector<double>& profit) {
        rows = static_cast<int>(limit.size());
        cols = static_cast<int>(profit.size());
        stride = cols + 2;
        tab.assign(static_cast<size_t>(rows + 2) * stride, 0.0);
        basis.resize(rows);
        nonbasis.resize(cols + 1);
        for (int i = 0; i < rows; ++i) {
            double* p = row(i);
            for (int j = 0; j < cols; ++j) p[j] = coef[i][j];
            p[cols] = -1.0; // 辅助变量  $x_0$  那一列, 只在找可行点时起作用
            p[cols + 1] = limit[i];
            basis[i] = cols + i; // 一开始所有变量取 0, 这一行剩的余量就是  $limit[i]$ 
        }
        double* obj = row(rows);
        // 目标行存的是  $-profit$ : 以后看到负数就说明 " 这个变量还能加, 目标还能变大 "
        for (int j = 0; j < cols; ++j) { obj[j] = -profit[j]; nonbasis[j] = j; }
        nonbasis[cols] = -1;
        row(rows + 1)[cols] = 1.0; // 找可行点那一阶段的目标: 把  $x_0$  压到 0
        weight.assign(cols + 1, 1.0);
        pivots = 0;
    }

    // 让第  $s$  列的变量从 0 开始往大加, 加到第  $r$  条约束把它卡住为止; 这两个变量交换身份
    void pivot(int r, int s) {
        ++pivots;
        double* pr = row(r);
        const double inv = 1.0 / pr[s];
        { // DeveX: 顺手更新参考权重, 供下次挑变量用 ( $O(cols)$ )
            const double as = pr[s], ws = weight[s];
            for (int j = 0; j <= cols; ++j) {
                if (j == s) continue;
                const double ratio = pr[j] / as;
                const double cand = ratio * ratio * ws;
                if (cand > weight[j]) weight[j] = cand;
            }
            const double nw = ws / (as * as);
            weight[s] = nw > 1.0 ? nw : 1.0;
        }
    }
}

```

```

const int total = rows + 2, width = cols + 2;
for (int i = 0; i < total; ++i) { // 用第 r 行把其余每行的第 s 列消成 0
    if (i == r) continue;
    double* pi = row(i);
    const double f = pi[s] * inv;
    if (f == 0.0) continue;
    for (int j = 0; j < width; ++j) pi[j] -= pr[j] * f; // 内层无分支, 编译器可向量化
    pi[s] = -f; // 第 s 列要单独修正
}
for (int j = 0; j < width; ++j) pr[j] *= inv;
pr[s] = inv;
swap(basis[r], nonbasis[s]);
}

// phase = 1: 先找一个满足所有约束的点 (把 x0 压到 0)
// phase = 2: 在可行的范围内把目标做到最大
bool run(int phase) {
    const int objRow = (phase == 1) ? rows + 1 : rows;
    // 目标值长时间不动 (退化) 就换 Bland 规则: 它慢, 但保证不会绕圈死循环
    long long stall = 0;
    const long long stallLimit = 4LL * (rows + cols) + 64;
    while (true) {
        const double* obj = row(objRow);
        int s = -1; // 先挑 " 加谁 ": 哪个变量还能让目标变大
        if (stall < stallLimit) { // Devez: 挑性价比最高的
            double best = 0.0;
            for (int j = 0; j <= cols; ++j) {
                if (phase == 2 && nonbasis[j] == -1) continue;
                const double d = obj[j];
                if (d > -eps) continue; // 不是负的 => 加它目标也不会变大
                const double score = d * d / weight[j];
                if (score > best) { best = score; s = j; }
            }
        } else { // Bland: 老实挑编号最小的
            for (int j = 0; j <= cols; ++j) {
                if (phase == 2 && nonbasis[j] == -1) continue;
                if (obj[j] < -eps && (s == -1 || nonbasis[j] < nonbasis[s])) s = j;
            }
        }
        if (s == -1) return true; // 没变量能让目标变大 => 这一阶段做完

        // 再看 " 加到哪停 ": 哪条约束最先被顶满, 即余量 / 系数 最小的那一行
        int r = -1;
        double bestRatio = 0.0, bestPiv = 0.0;
        for (int i = 0; i < rows; ++i) {
            const double* pi = row(i);
            if (pi[s] < eps) continue; // 系数 <= 0, 这条约束拦不住它
            const double ratio = pi[cols + 1] / pi[s];
            if (r == -1 || ratio < bestRatio - 1e-12 ||
                // 两行一样紧时选系数绝对值更大的那行, 算起来数值更稳
                (ratio < bestRatio + 1e-12 && pi[s] > bestPiv)) {
                r = i; bestRatio = ratio; bestPiv = pi[s];
            }
        }
        if (r == -1) return false; // 没人拦得住它 => 目标能到无穷大

        const double before = row(objRow)[cols + 1];
        pivot(r, s);
        const double after = row(objRow)[cols + 1];
        stall = (after > before + 1e-12) ? 0 : stall + 1;
    }
}

```

```

// 返回目标能达到的最大值；无界返回 +inf，一个可行点都没有返回 -inf
// 传了 solution 就顺便把最优时每个变量的取值填回去
double solve(vector<double>* solution = nullptr) {
    int r = 0;
    for (int i = 1; i < rows; ++i)
        if (row(i)[cols + 1] < row(r)[cols + 1]) r = i;
    if (row(r)[cols + 1] < -eps) { // 有约束上限是负的 => 全取 0 都不合法，
        ↪ 先找可行点
        pivot(r, cols);
        if (!run(1) || row(rows + 1)[cols + 1] < -eps)
            return -numeric_limits<double>::infinity(); // x0 压不到 0 => 无解
        for (int i = 0; i < rows; ++i) {
            if (basis[i] != -1) continue; // 把还赖在解里的 x0 换出去
            int s = -1;
            for (int j = 0; j <= cols; ++j)
                if (s == -1 || row(i)[j] < row(i)[s] ||
                    (row(i)[j] == row(i)[s] && nonbasis[j] < nonbasis[s])) s = j;
            pivot(i, s);
        }
    }
    if (!run(2)) return numeric_limits<double>::infinity();
    if (solution) { // 没被解出来的变量取值就是 0
        solution->assign(cols, 0.0);
        for (int i = 0; i < rows; ++i)
            if (basis[i] < cols) (*solution)[basis[i]] = row(i)[cols + 1];
    }
    return row(rows)[cols + 1];
}
};

```

7 位运算相关恒等式

在处理位运算相关的数学推导时，往往需要从其本质意义出发来理解其与四则运算的关系。为了书写清晰，以下使用 $\oplus$ 表示按位异或，使用 $\&$ （即 $\&$ ）表示按位与，使用 $|$ （即 $|$ ）表示按位或，使用 NOT 表示按位取反。

7.1 加法与位运算转换

$$a + b = (a \oplus b) + 2 \times (a \& b)$$

意义解释： $a \oplus b$ 本质上是“不进位加法”，它计算了每一位上独立相加的结果；而 $a \& b$ 则找出了所有产生进位的位。因为进位会向高位进一（即在数值上相当于乘以 2），所以真实加法等于不进位加法加上两倍的进位值。

推广到三个数：令 $S = (X \oplus Y) \& Z + X \& Y$ ，则

$$X + Y + Z = (X \oplus Y \oplus Z) + 2S$$

推导：先对 X, Y 使用两数恒等式得 $X + Y = (X \oplus Y) + 2(X \& Y)$ ，再令 $a = X \oplus Y$ ，对 a 与 Z 使用恒等式得 $a + Z = (a \oplus Z) + 2(a \& Z)$ ，合并即得。

S 的两项没有公共位：第一阶段 $X + Y$ 产生进位 $X \& Y$ ，同时把这些位在异或残差 $X \oplus Y$ 中清零了（两个 1 对冲变为 0）；第二阶段拿残差与 Z 相加，但那些位已经是 0， $(X \oplus Y) \& Z$ 在那里不可能为 1。因此两项的加法等于按位或，不会再产生新的进位，即 $S = (X \oplus Y) \& Z | (X \& Y)$ ， S 是逐位运算的结果。

应用场景：当问题同时约束三个数的加法和与异或和时（如 CodeChef “Sum and XOR”），该恒等式可将全局加法约束分解为逐位独立的计数问题。

7.2 或、与、异或的关系

$$a | b = (a \oplus b) + (a \& b)$$

意义解释： $a | b$ 表示两数中至少有一个为 1 的位。这些位可以拆分成两类：一类是只有其中一个数为 1 的位（即 $a \oplus b$ ），另一类是两个数都为 1 的位（即 $a \& b$ ）。由于这两类位的集合互不相交，因此它们的加法可以直接等价于按位或（相加过程没有任何进位）。

推论：将上式代入加法等式，可得两数之和也等于它们的按位或与按位与之和：

$$a + b = (a | b) + (a \& b)$$

$a | b$ 相当于丢失了需要进位的位，当然，相比于异或，只丢失了一份。因此就加上一份这个 $a \& b$ 就可以了。

下面这个就是一个移项：

$$a \oplus b = (a | b) - (a \& b)$$

7.3 减法与位运算

$$a - b = (a \oplus b) - 2 \times ((\text{NOT } a) \& b)$$